

Evaluation Measures

- Are we doing well? Is System A doing better than System B?
- A measure of how (in)accurate a system is on a task
 - In many cases Error (accuracy/PC) is not the best measure
 - Using the appropriate measure will help select the best algorithm

Evaluation Measures

- Are we doing well? Is System A doing better than System B?
- A measure of how (in)accurate a system is on a task
 - In many cases Error (accuracy/PC) is not the best measure
 - Using the appropriate measure will help select the best algorithm
- **Classification**
 - How often we classify something right/wrong
- Regression
 - How close are we to what we are trying to predict
- Unsupervised
 - How well do we describe our data
 - In general – very hard

Evaluation Measures

- Are we doing well? Is System A doing better than System B?
- A measure of how (in)accurate a system is on a task
 - In many cases Error (accuracy/PC) is not the best measure
 - Using the appropriate measure will help select the best algorithm
- Classification
 - How often we classify something right/wrong
- **Regression**
 - How close are we to what we are trying to predict
- Unsupervised
 - How well do we describe our data
 - In general – very hard

Evaluation Measures

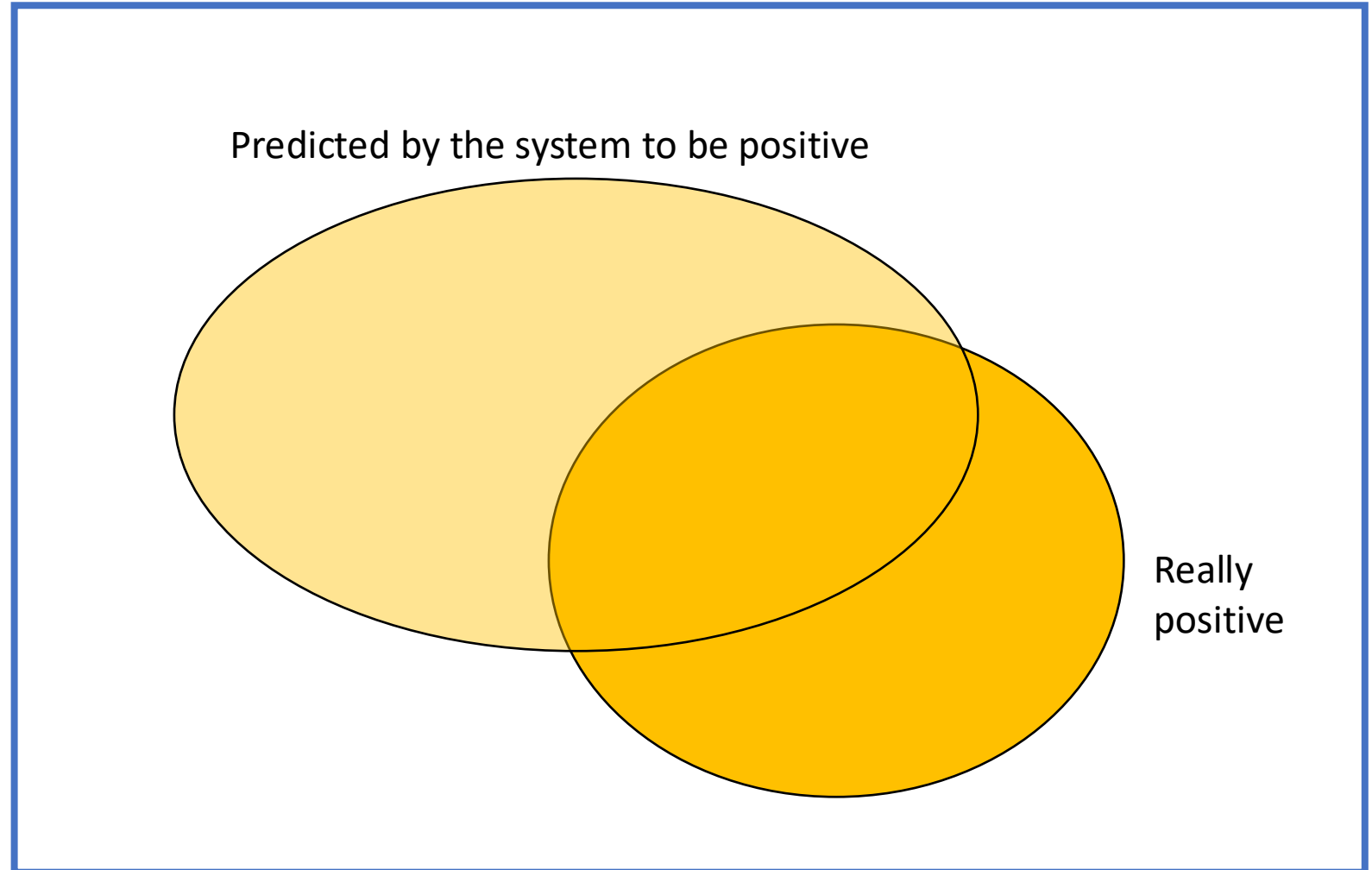
- Are we doing well? Is System A doing better than System B?
- A measure of how (in)accurate a system is on a task
 - In many cases Error (accuracy/PC) is not the best measure
 - Using the appropriate measure will help select the best algorithm
- Classification
 - How often we classify something right/wrong
- Regression
 - How close are we to what we are trying to predict
- **Unsupervised**
 - How well do we describe our data
 - In general – very hard

Model Skill

- Once you have established the measures you are going to use for evaluation then we can consider this our model **skill**.
- ***Thereafter we are looking to find the machine learning approach that maximises its “skill”***

Classification Measures: Basics

All testing instances

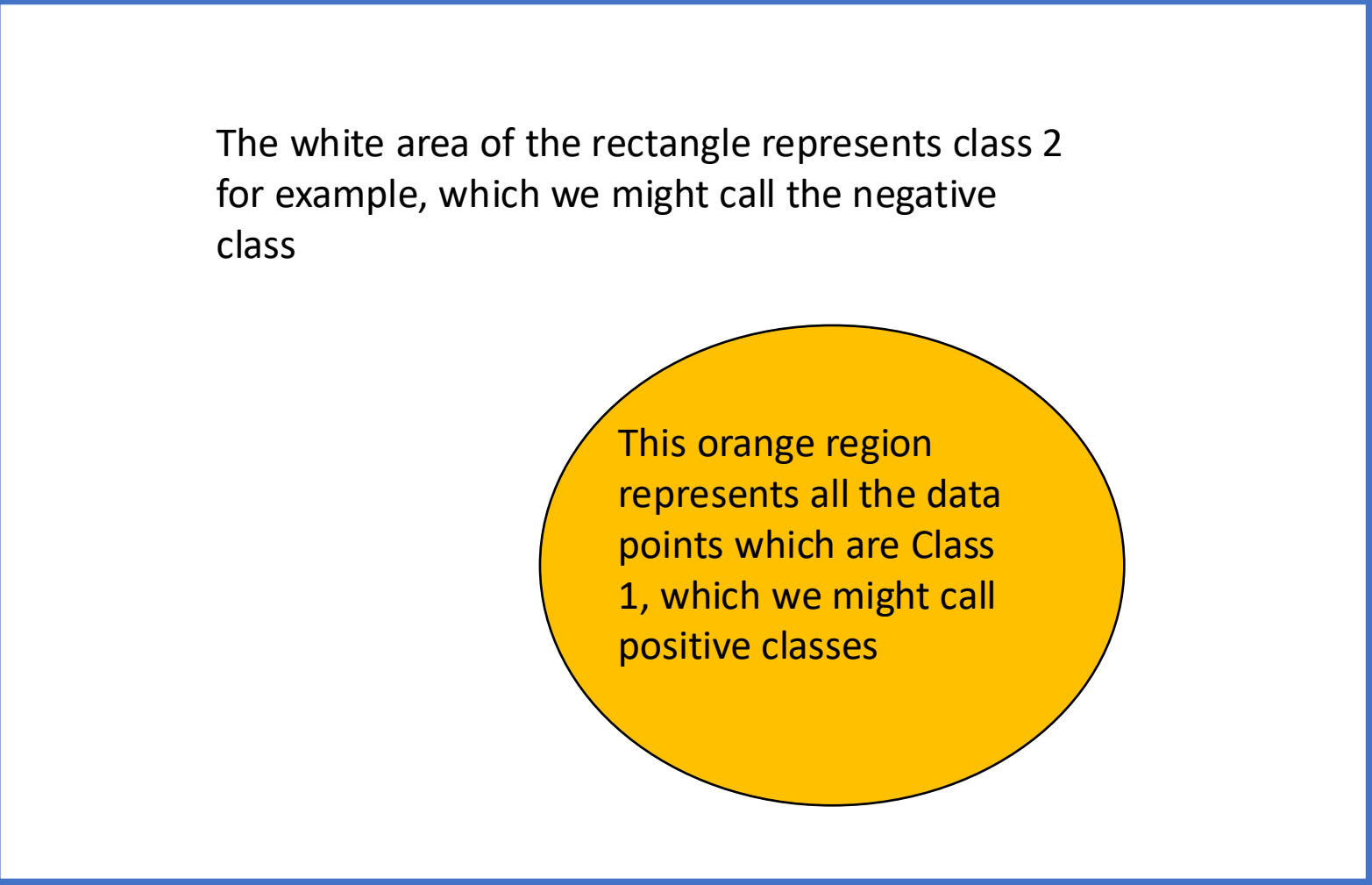


Aside: In this example we are talking about 2-class (binary) classification...so if it makes sense for the problem you can call one class the positive class and the other the negative class. You choose which class you call positive.

Classification Measures: Basics

The rectangle represents all testing instances

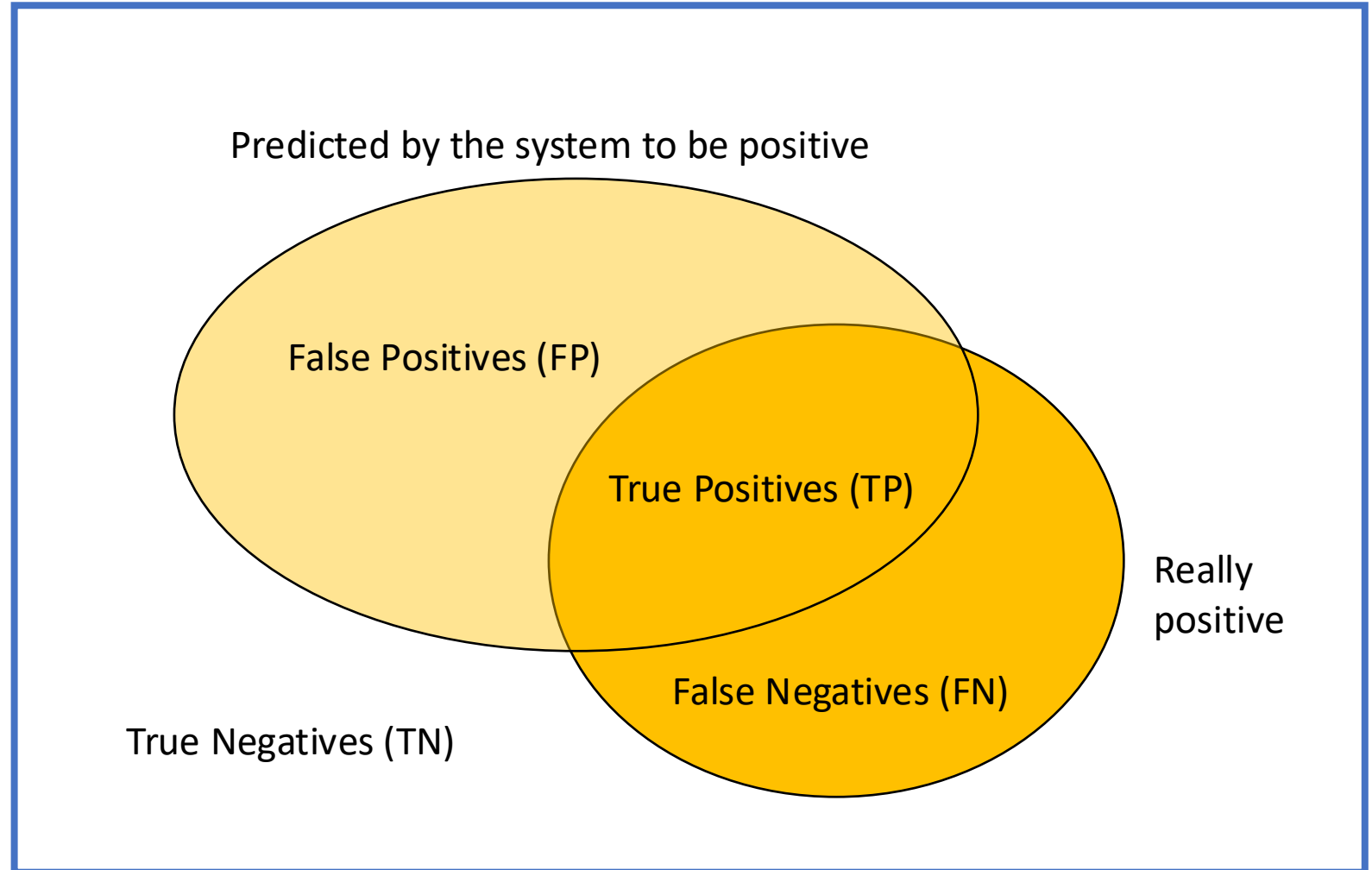
The white area of the rectangle represents class 2 for example, which we might call the negative class



This orange region represents all the data points which are Class 1, which we might call positive classes

Classification Measures: Basics

All testing instances



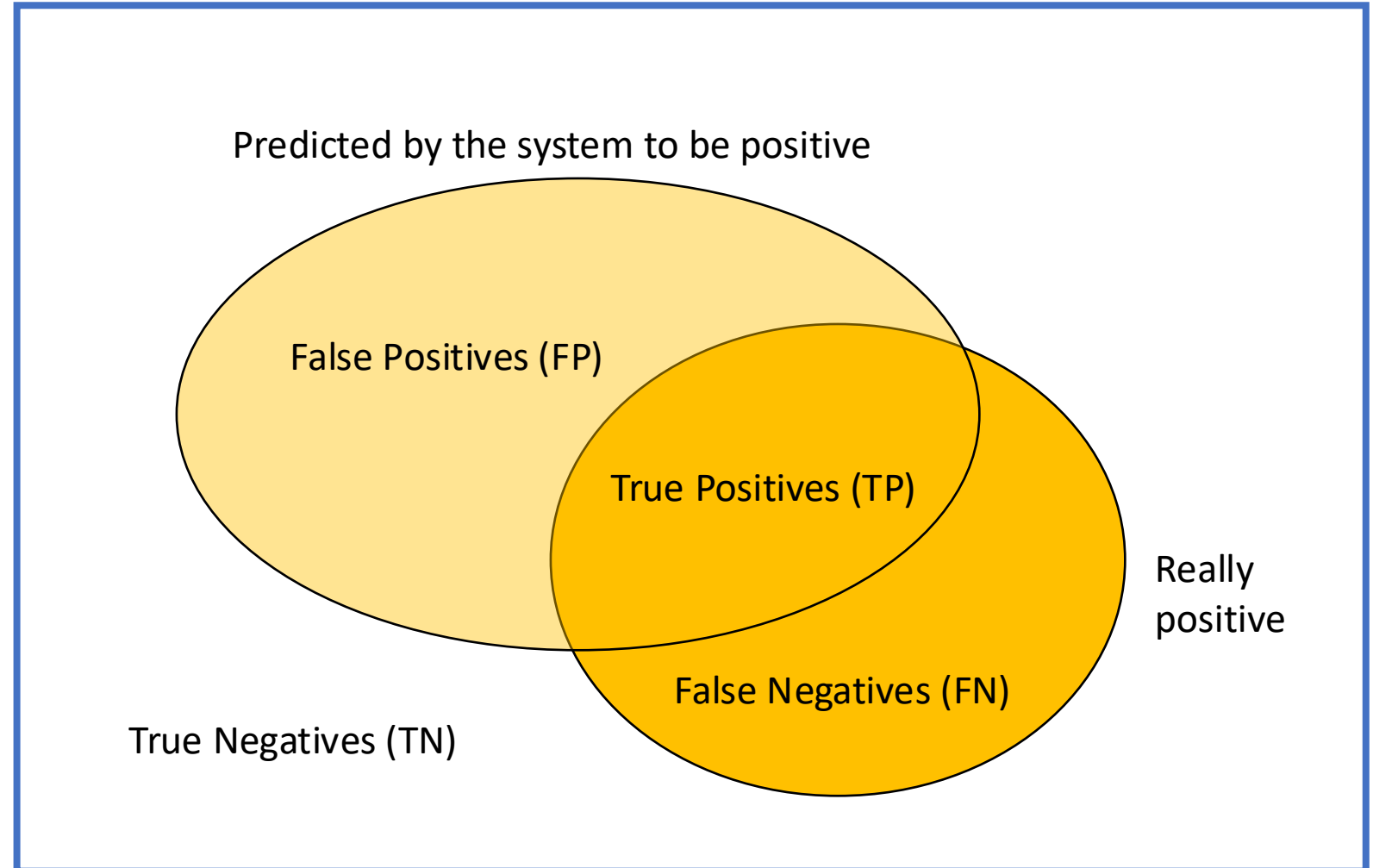
Classification Measures: Basics

All testing instances

	Predict Positive ?	
	Yes	No
Really Positive?		
Yes	TP	FN
No	FP	TN

Confusion Matrix for 2-class
classification

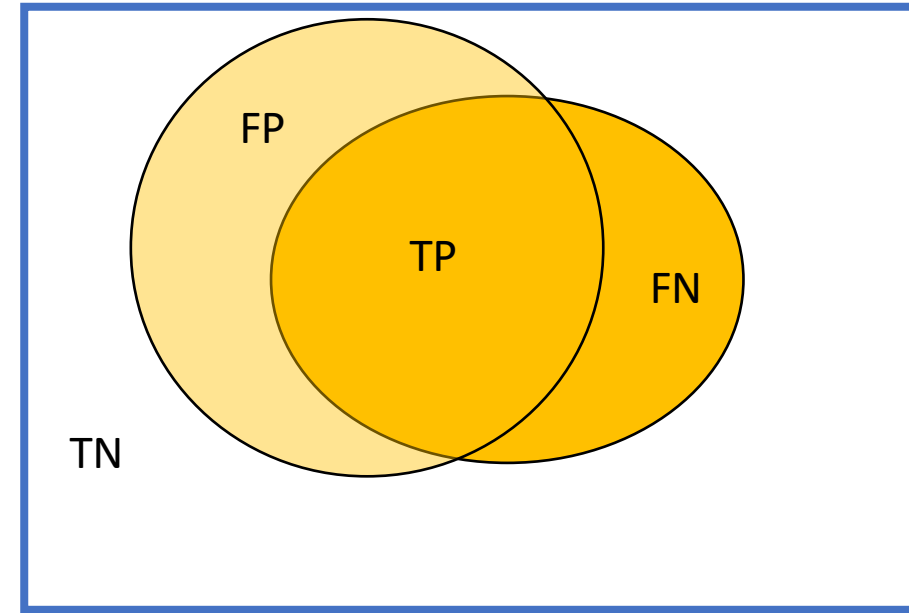
Want large diagonal with small
FP and FN



Classification Error

- Classification Error = $\frac{\text{Errors}}{\text{Total}} = \frac{FP+FN}{TP+TN+FP+FN}$

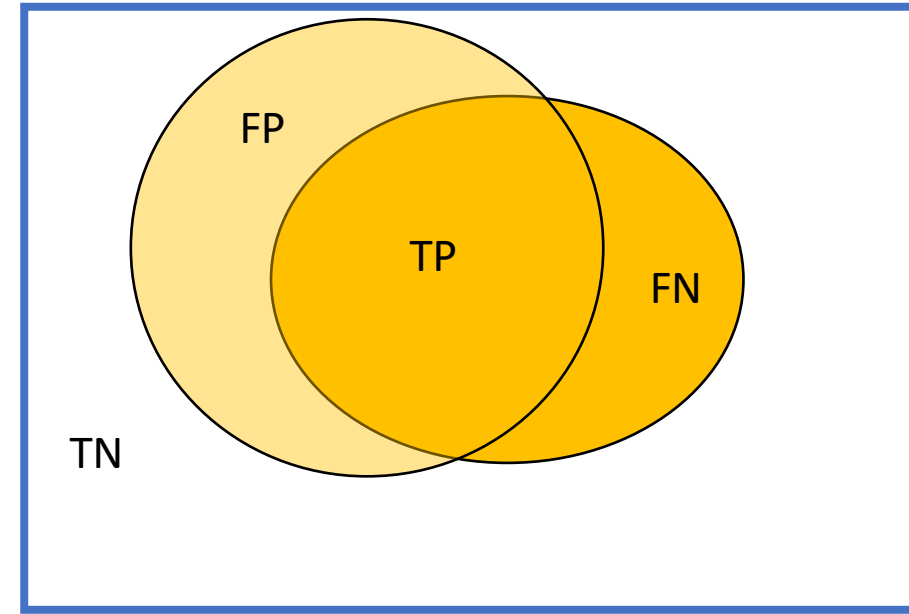
- Accuracy = $(1 - \text{Error}) = \frac{\text{correct}}{\text{total}} = \frac{TP+TN}{TP+TN+FP+FN}$



Classification Error

- Classification Error = $\frac{\text{Errors}}{\text{Total}} = \frac{FP+FN}{TP+TN+FP+FN}$

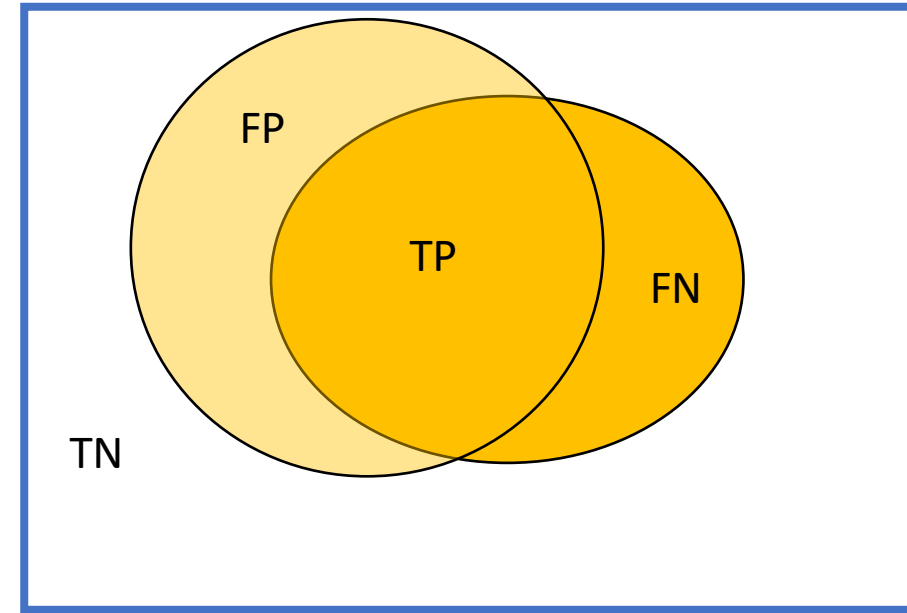
All the "Falses",
i.e. errors



- Accuracy = (1 - Error) = $\frac{\text{correct}}{\text{total}} = \frac{TP+TN}{TP+TN+FP+FN}$

Classification Error

- Classification Error = $\frac{\text{Errors}}{\text{Total}} = \frac{FP + FN}{TP + TN + FP + FN}$
 - All the "Falses",
i.e. errors
 - All the "Decisions",
i.e. total predictions made
- Accuracy = (1 - Error) = $\frac{\text{correct}}{\text{total}} = \frac{TP + TN}{TP + TN + FP + FN}$

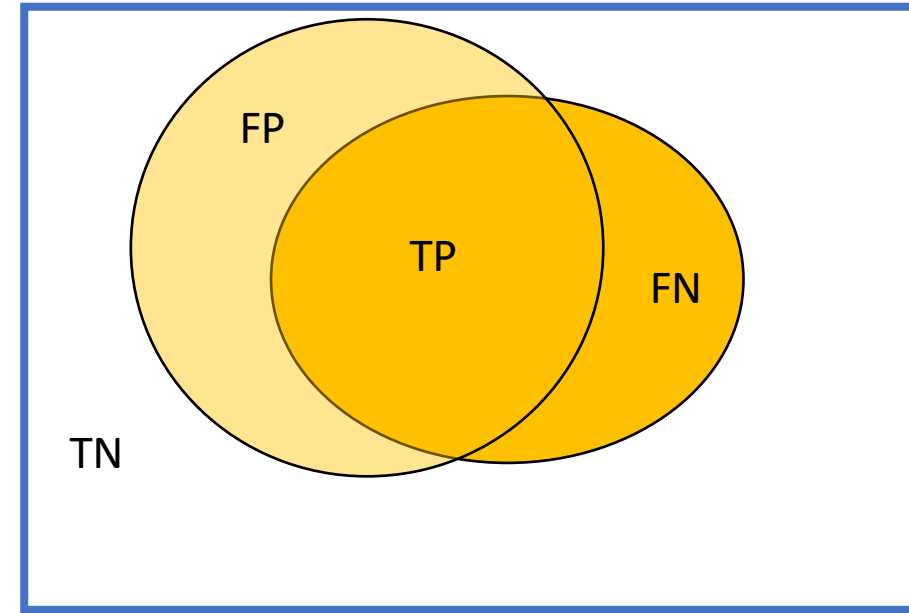


Classification Error

- Classification Error = $\frac{Errors}{Total} = \frac{FP+FN}{TP+TN+FP+FN}$

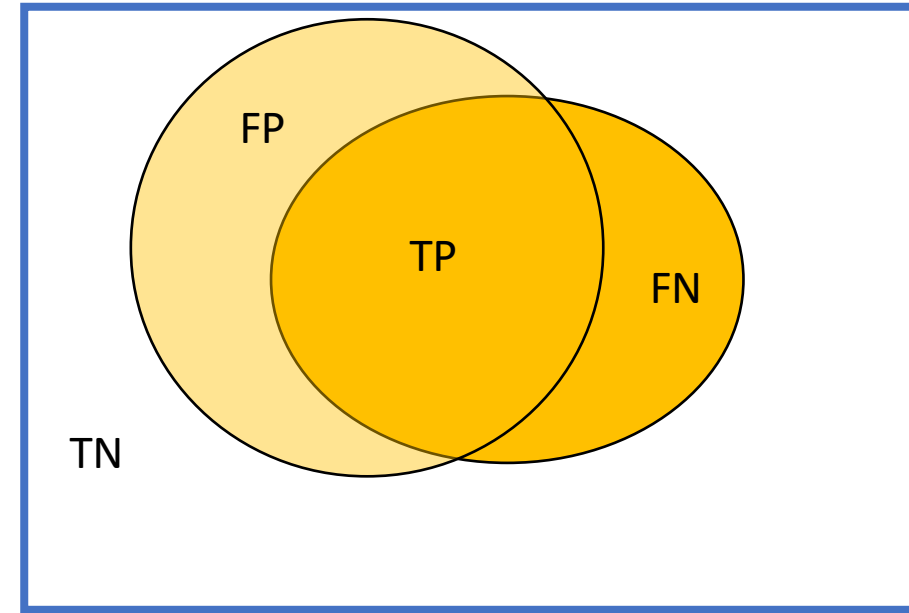
- Accuracy = (1 - Error) = $\frac{\text{correct}}{total} = \frac{TP+TN}{TP+TN+FP+FN}$

All the "Trues", i.e.
correct decisions



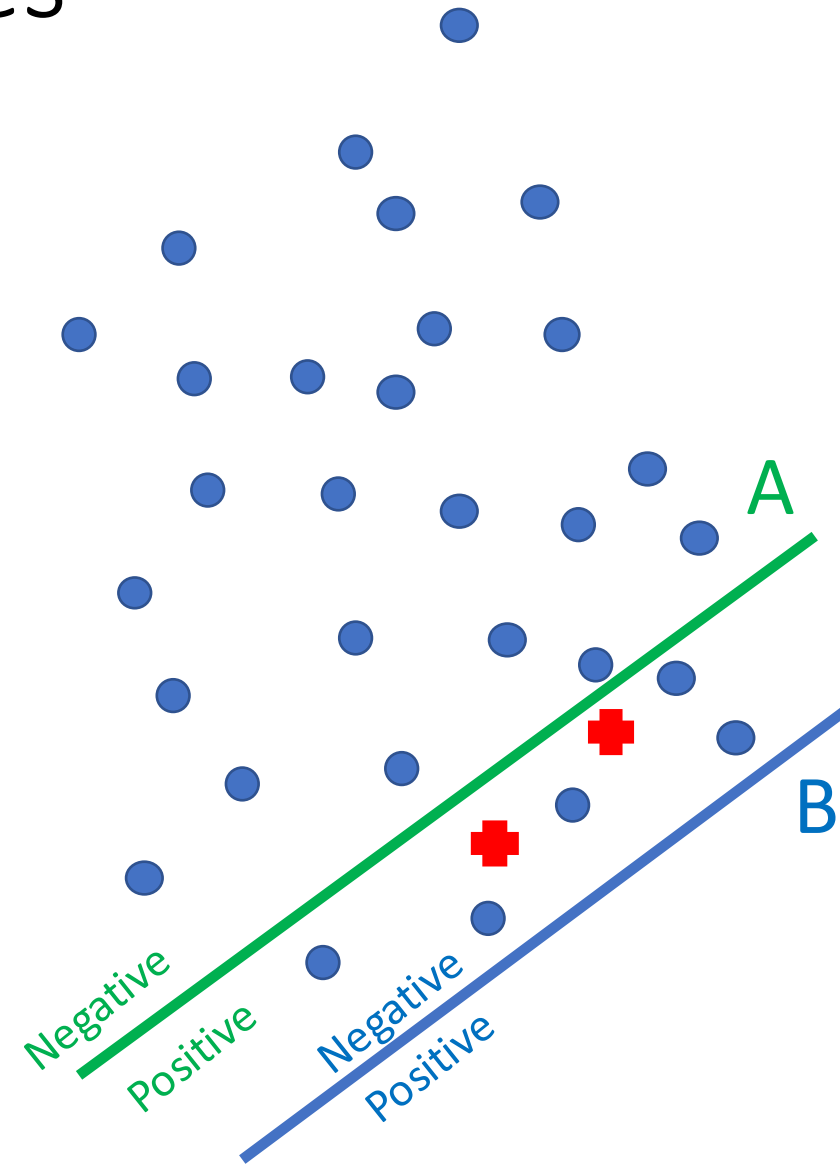
Classification Error

- Classification Error = $\frac{\text{Errors}}{\text{Total}} = \frac{FP+FN}{TP+TN+FP+FN}$
- Accuracy = $(1 - \text{Error}) = \frac{\text{correct}}{\text{total}} = \frac{TP+TN}{TP+TN+FP+FN}$
- Basic measure of goodness of classifier
- Problem: Misleading if we have unbalanced classes
 - Example 1: Predict whether an earthquake is about to happen
 - Happen very rarely, very good accuracy if always predict "No"
 - Solution: Make FN much more "costly" than FPs.
 - Example 2: Websearch – decided if a webpage is relevant to a user
 - 99.9999% of pages not relevant to a query so return nothing
 - Solution: Use measures that don't involve TN (such as recall/ precision).



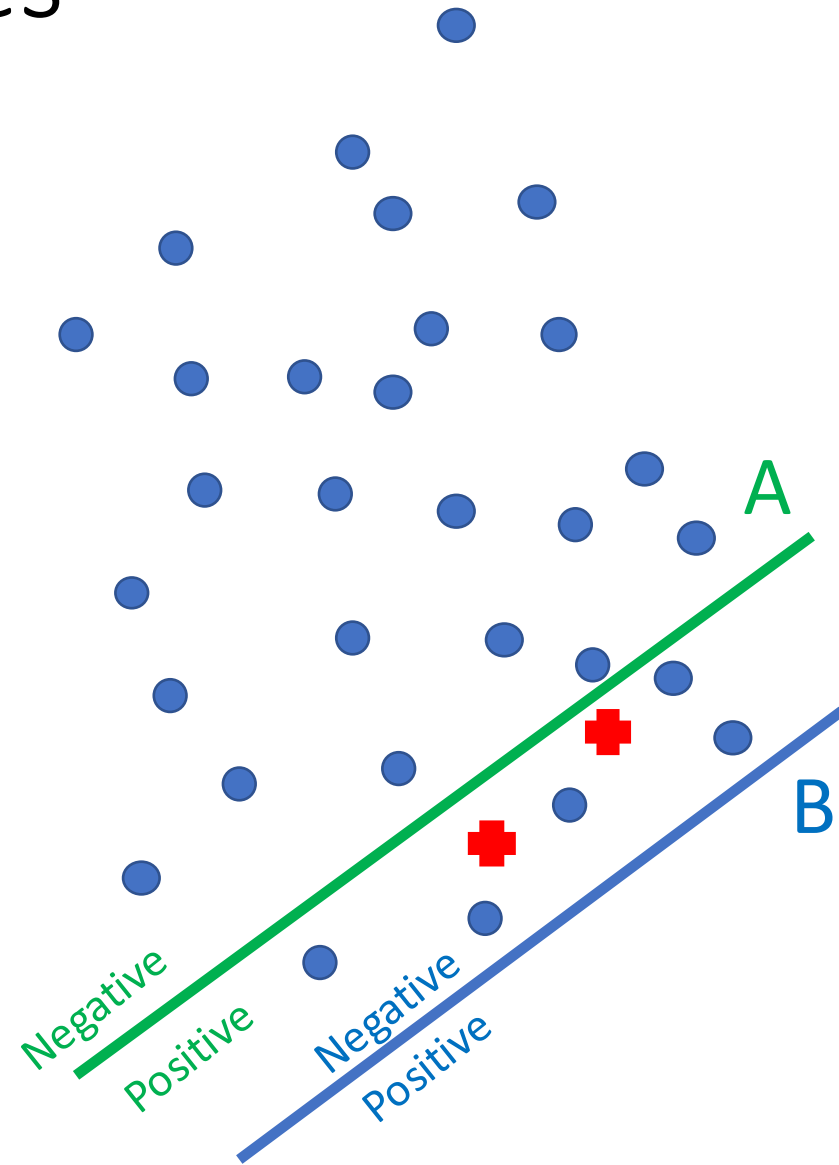
Accuracy and Unbalanced Classes

- Lets say you are predicting 1 billion+ Youtube videos, i.e. + versus normal Youtube content labelled as ●



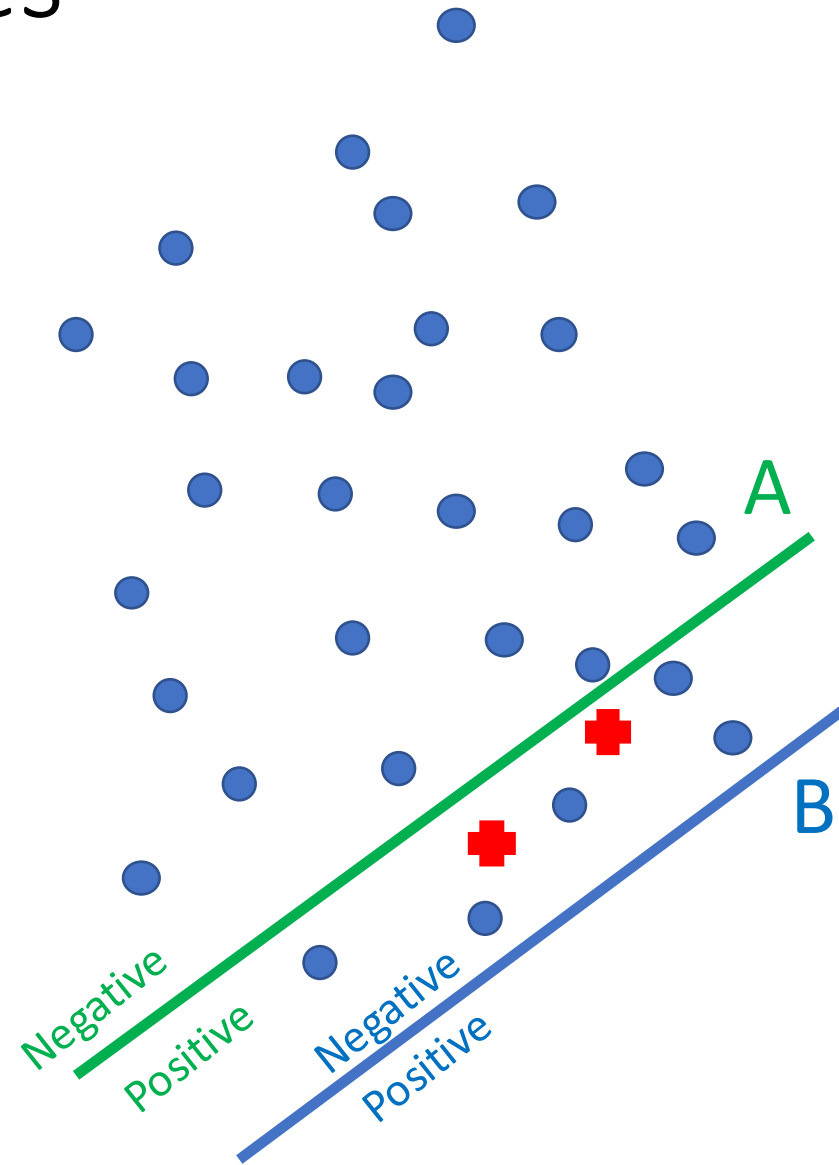
Accuracy and Unbalanced Classes

- Humans would prefer a classifier that gives the decision boundary marked as A



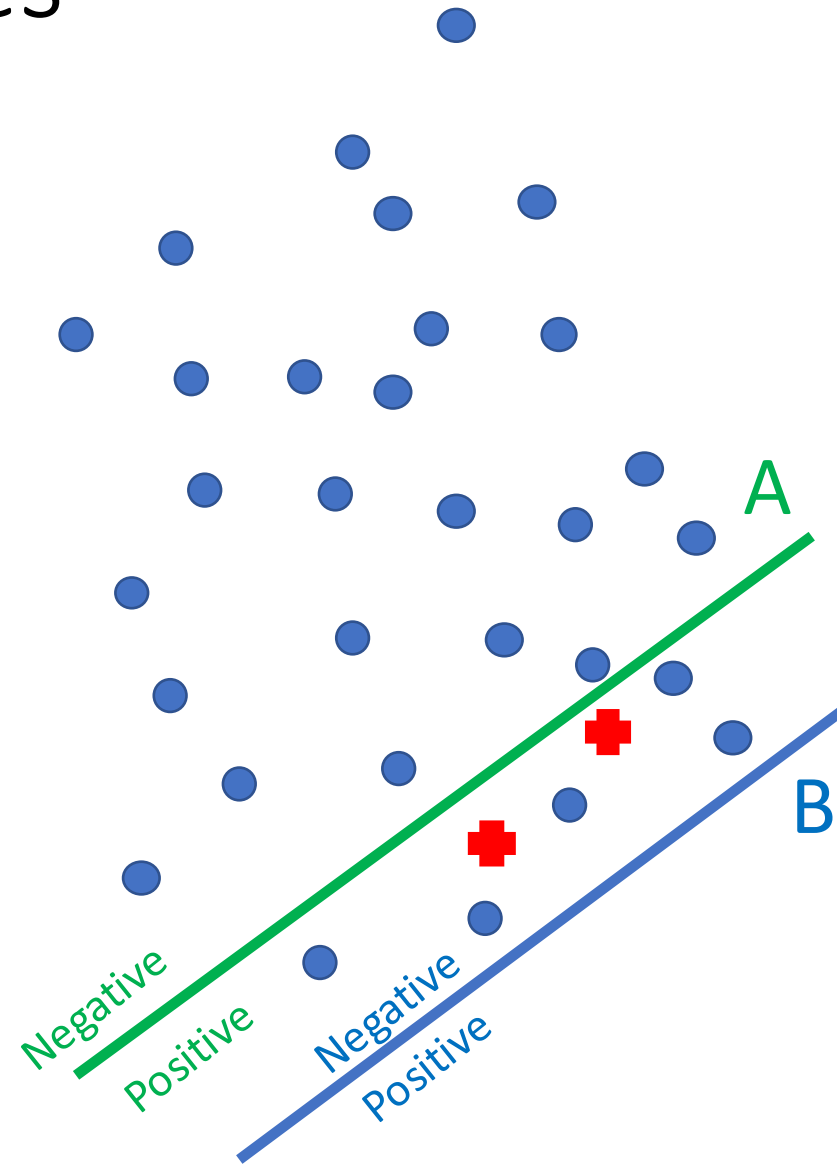
Accuracy and Unbalanced Classes

- Humans would prefer a classifier that gives the decision boundary marked as A
- Accuracy will lead to the selection of a classifier which produces decision boundary B (Classifier A makes 5 errors, Classifier B makes 2 errors)



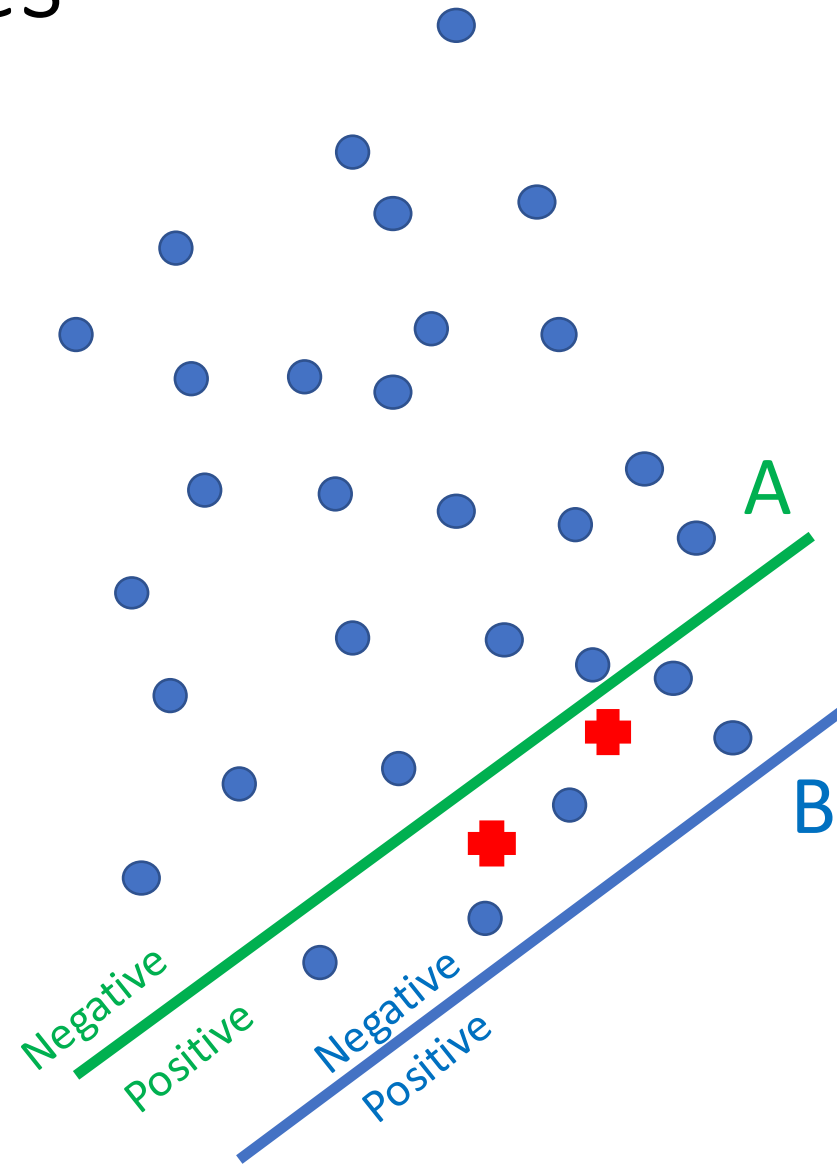
Accuracy and Unbalanced Classes

So the pursuit of “Accuracy” leads to a suboptimal choice of classifier, i.e. we have selected the wrong classifier.



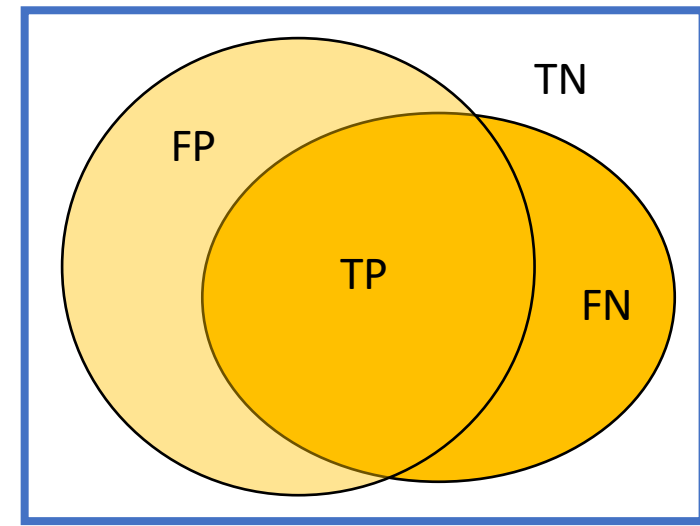
Accuracy and Unbalanced Classes

- A good article reviewing the issues associated with and fixes for imbalanced datasets can be found here <https://github.com/pb111/Data-Preprocessing-Project-Imbalanced-Classes-Problem>)



Misses and False Alarms

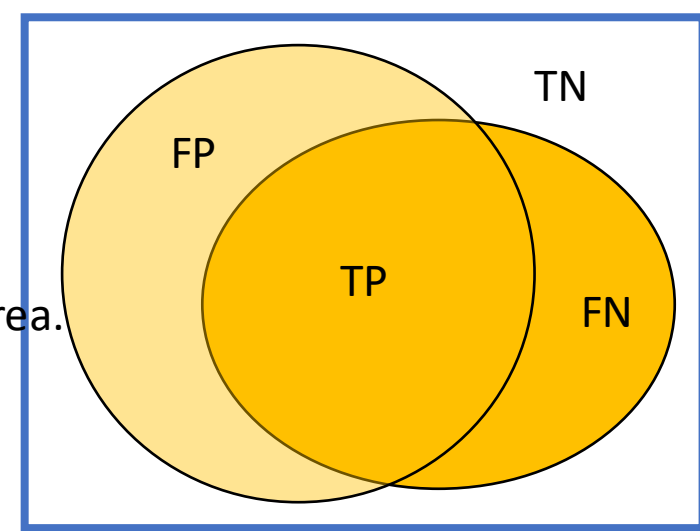
- False alarm rate = False positive rate = $FP/(FP+TN)$
 - % of negatives we misclassified as positives



Misses and False Alarms

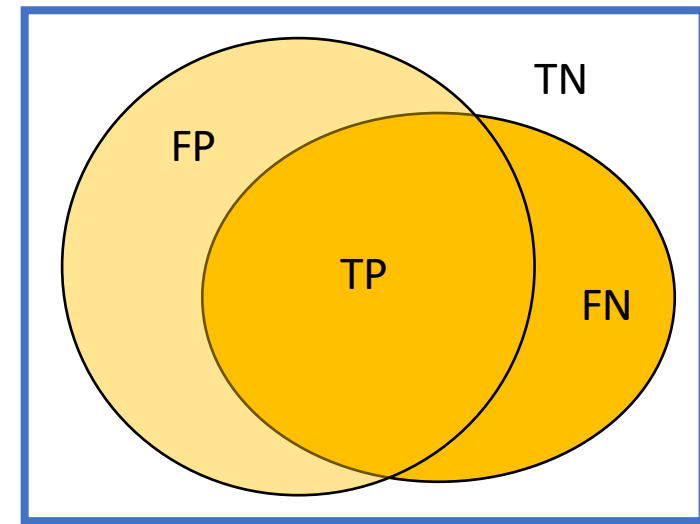
- False alarm rate = False positive rate = $FP / (FP + TN)$
 - % of negatives we misclassified as positives

All the negative instances, the original white area.



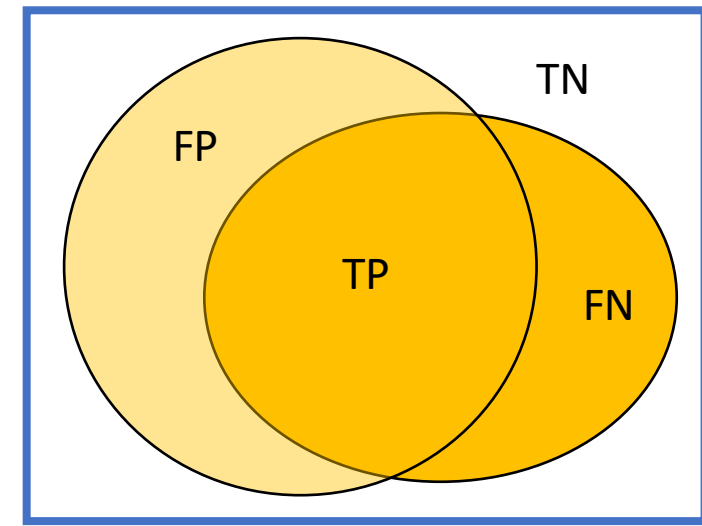
Misses and False Alarms

- False alarm rate = False positive rate = $FP/(FP+TN)$
 - % of negatives we misclassified as positives
- Miss rate = False negative rate = $FN/(TP+FN)$
 - % of positives we misclassified as negatives



Misses and False Alarms

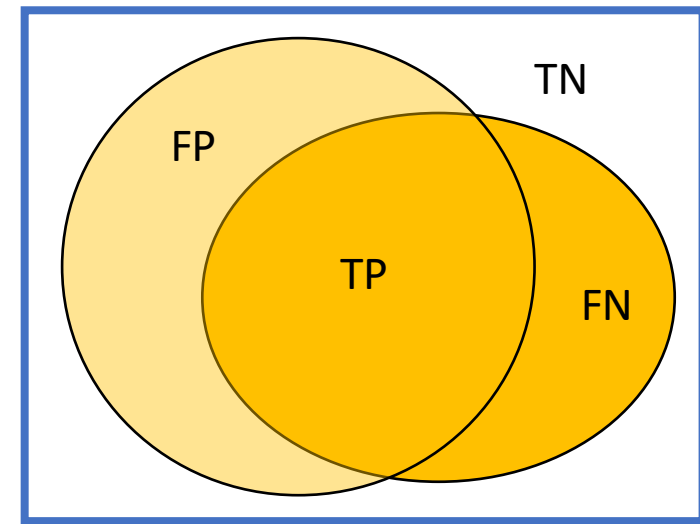
- False alarm rate = False positive rate = $FP / (FP + TN)$
 - % of negatives we misclassified as positives
- Miss rate = False negative rate = $FN / (TP + FN)$
 - % of positives we misclassified as negatives



All the positive instances, the original orange region.

Misses and False Alarms

- False alarm rate = False positive rate = $FP/(FP+TN)$
 - % of negatives we misclassified as positives
- Miss rate = False negative rate = $FN/(TP+FN)$
 - % of positives we misclassified as negatives
- **Recall = True Positive Rate = $TP/(TP+FN)$**
 - % of positives we correctly identify (1- Miss Rate)

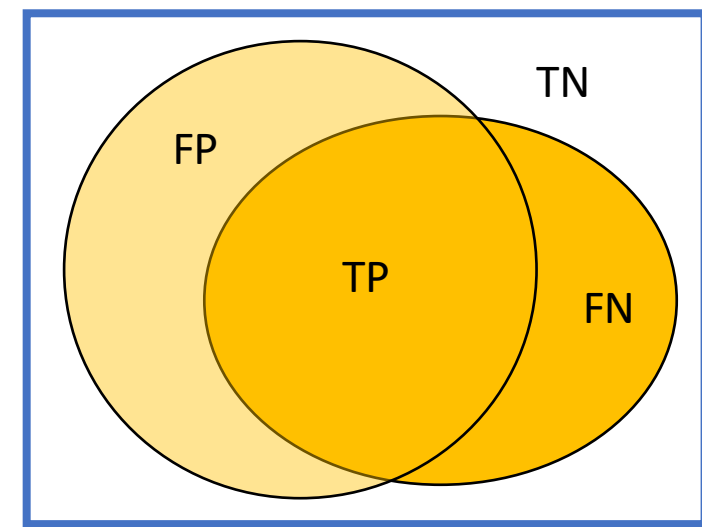


Misses and False Alarms

- False alarm rate = False positive rate = $FP/(FP+TN)$
 - % of negatives we misclassified as positives
- Miss rate = False negative rate = $FN/(TP+FN)$
 - % of positives we misclassified as negatives
- **Recall = True Positive Rate = $TP/(TP+FN)$**
 - % of positives we correctly identify (1- Miss Rate)
- **Precision = $TP/(TP+FP)$**
 - % positive out of what we predicted was positive

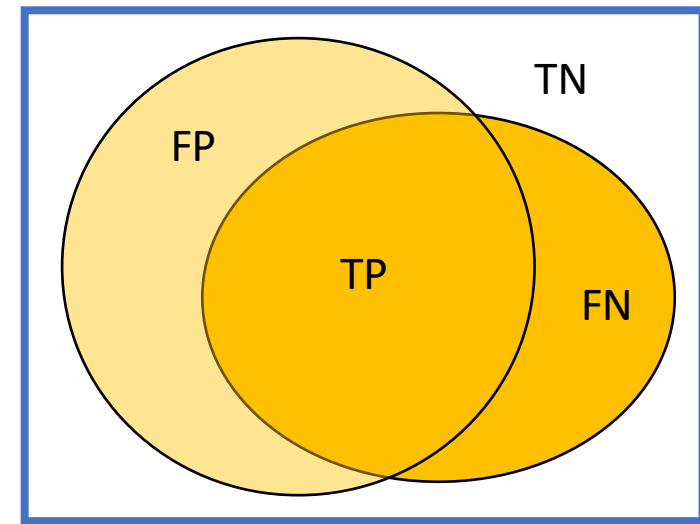
Precision is useful – it is a measure of given that everything our system thought was positive what fraction of these were actually positive?

Have a think about this say for imbalanced classes - make sure you can tell the difference between recall and precision.



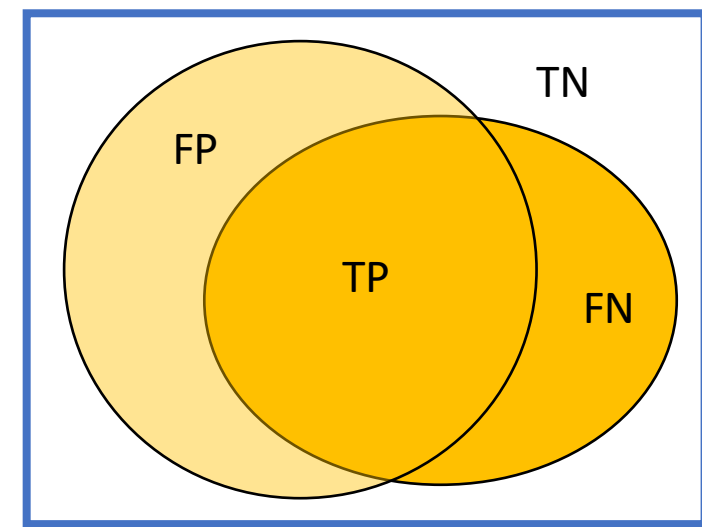
Misses and False Alarms

- False alarm rate = False positive rate = $FP/(FP+TN)$
 - % of negatives we misclassified as positives
- Miss rate = False negative rate = $FN/(TP+FN)$
 - % of positives we misclassified as negatives
- Recall = True Positive Rate = $TP/(TP+FN)$
 - % of positives we correctly identify (1- Miss Rate)
- Precision = $TP/(TP+FP)$
 - % positive out of what we predicted was positive
- Meaningless to report just one of these
 - Trivial to get 100% recall or 0% false alarm rate
 - Typical: Recall/Precision or Miss/FA rate or TP/FP rate



Misses and False Alarms

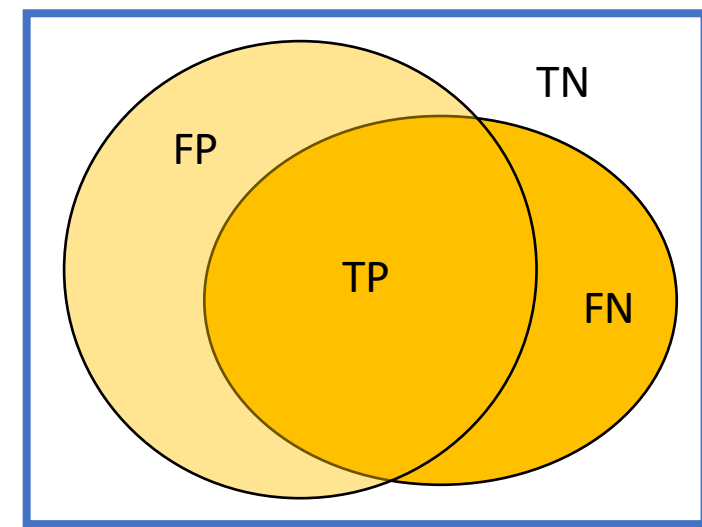
- False alarm rate = False positive rate = $FP/(FP+TN)$
 - % of negatives we misclassified as positives
- Miss rate = False negative rate = $FN/(TP+FN)$
 - % of positives we misclassified as negatives
- Recall = True Positive Rate = **$TP/(TP+FN)$**
 - % of positives we correctly identify (1- Miss Rate)
- Precision = $TP/(TP+FP)$
 - % positive out of what we predicted was positive
- Meaningless to report just one of these
 - Trivial to get 100% recall or 0% false alarm rate
 - Typical: Recall/Precision or Miss/FA rate or TP/FP rate



Just predict everything to be the positive class. You will catch every positive instance. Example seizure detection algorithm that ends up being switched off by clinicians

Misses and False Alarms

- False alarm rate = False positive rate = $FP/(FP+TN)$
 - % of negatives we misclassified as positives
- Miss rate = False negative rate = $FN/(TP+FN)$
 - % of positives we misclassified as negatives
- Recall = True Positive Rate = $TP/(TP+FN)$
 - % of positives we correctly identify (1- Miss Rate)
- Precision = $TP/(TP+FP)$
 - % positive out of what we predicted was positive
- Meaningless to report just one of these
 - Trivial to get 100% recall or 0% false alarm rate
 - Typical: Recall/Precision or Miss/FA rate or TP/FP rate



Just say “No” to everything, i.e. predict everything to be the negative class. You will never have a false alarm but you will never catch any positive instances either. Think of seizure alarm example again, what would this mean?

Sensitivity and Specificity – Common in Health Applications

Sensitivity addresses **the Question**: "Out of everyone who is *actually* positive, how many did we find?"

Alternative Name: True Positive Rate (TPR) or Recall.

- **Focus**: Avoiding False Negatives (missing cases).

- **Formula**:

$$\text{Sensitivity} = \frac{TP}{TP + FN}$$

	Disease present	Disease absent
Positive test	True positive	False positive
Negative test	False negative	True negative

- **Analogy**: A metal detector. High sensitivity means it beeps for every bit of metal (even a paperclip). It rarely "misses" metal.

Sensitivity and Specificity – Common in Health Applications

Specificity addresses **the Question**: "Out of everyone who is *actually* negative, how many did we correctly ignore?"

Alternative Name: True Negative Rate (TPR).

- **Focus**: Avoiding False Positives. (False alarms).
- **Formula**:

$$\text{Specificity} = \frac{TN}{TN + FP}$$

	Disease present	Disease absent
Positive test	True positive	False positive
Negative test	False negative	True negative

- **Analogy**: A high-end security guard who only stops people who are actually on the blacklist. High specificity means they don't harass innocent people.

PPV (Positive Predictive Value / Precision)

The Question: "If the model says 'Positive,' how likely is it that the person *is* actually positive?"

Focus: Reliability of a positive alert.

- **Formula:**

$$\text{PPV} = \frac{TP}{TP + FP}$$

- **Crucial Note:** PPV is heavily influenced by **Prevalence**. If a disease is very rare, even a great model will have a low PPV because most "positive" hits will be rare false alarms.

NPV (Negative Predictive Value)

The Question: "If the model says 'Negative,' how likely is it that the person *is* actually negative?"

Focus: Reliability of a "clear" result.

- **Formula:**

$$\text{NPV} = \frac{TN}{TN + FN}$$

- **Crucial Note:** If a disease is extremely common, NPV will drop because the model is more likely to accidentally let a positive case slip through as a negative.

Summary of these

Metric	Viewpoint	Main Concern	Example Scenario
Sensitivity	Actual Positives	"Don't miss the sick"	Cancer screening (rather have a false alarm than miss it).
Specificity	Actual Negatives	"Don't annoy the healthy"	Spam filters (don't send a work email to the trash).
PPV	Predicted Positives	"Is this 'Yes' real?"	A pregnancy test result in your hand.
NPV	Predicted Negatives	"Is this 'No' real?"	Being told you are "Clear" after a drug test.

Worth Noting

- **Sensitivity/Specificity are properties of the model, but PPV/NPV are properties of the population.** If you take the same model from a hospital (high prevalence) to a general school (low prevalence), your PPV will plummet even though the model didn't change!

PPV/NPV vs Prevalence

Prevalence	PPV	NPV
1%	8%	>99%
10%	50%	99%
20%	69%	97%
50%	90%	90%

Utility and Cost

- Sometimes need a single number evaluation measure
 - Optimizing the learner automatically, competitive evaluation
 - May know costs of different errors, e.g. earthquake example
 - False positive: cost of preventative measures (evacuation, loss profit ,etc)
 - False negative: cost of recovery (reconstruction, liability)

Utility and Cost

- Sometimes need a single number evaluation measure
 - Optimizing the learner automatically, competitive evaluation
 - May know costs of different errors, e.g. earthquake example
 - False positive: cost of preventative measures (evacuation, loss profit ,etc)
 - False negative: cost of recovery (reconstruction, liability)
- Detection cost: weighted average of FP, FN rates
 - $\text{Cost} = C_{\text{FP}} * \text{FP} + C_{\text{FN}} * \text{FN}$ [event detection]
- F-measure: harmonic mean of recall and precision
 - $F1 = 2 / (1/\text{Recall} + 1/\text{Precision})$ [Information Retrieval]

Thresholds in Classification

- Two systems have the following performance
 - System A: True Positive = 50% , False positive = 20%
 - System B: True Positive = 100%, False positive = 60%

Thresholds in Classification

- Two systems have the following performance
 - System A: True Positive = 50% , False positive = 20%
 - System B: True Positive = 100%, False positive = 60%
- Which system is better? (assume no apriori utility)
 - Misleading question
 - A and B could be the exact same system operating at different thresholds

ROC Curves

- Many algorithms compute “confidence” $f(x)$
 - threshold to get decision: spam if $f(x) > t$, non-spam if $f(x) \leq t$
 - Naïve Bayes: $P(\text{spam}|x) > 0.5$, Linear/Logistic/SVM: $w^T x > 0$, Decision Tree: $p_+/p_- > 1$
 - threshold t determines error rates
 - False Positive rate = $P(f(x) > t | \text{ham})$, True Positive rate = $P(f(x) > t | \text{spam})$

ROC Curves

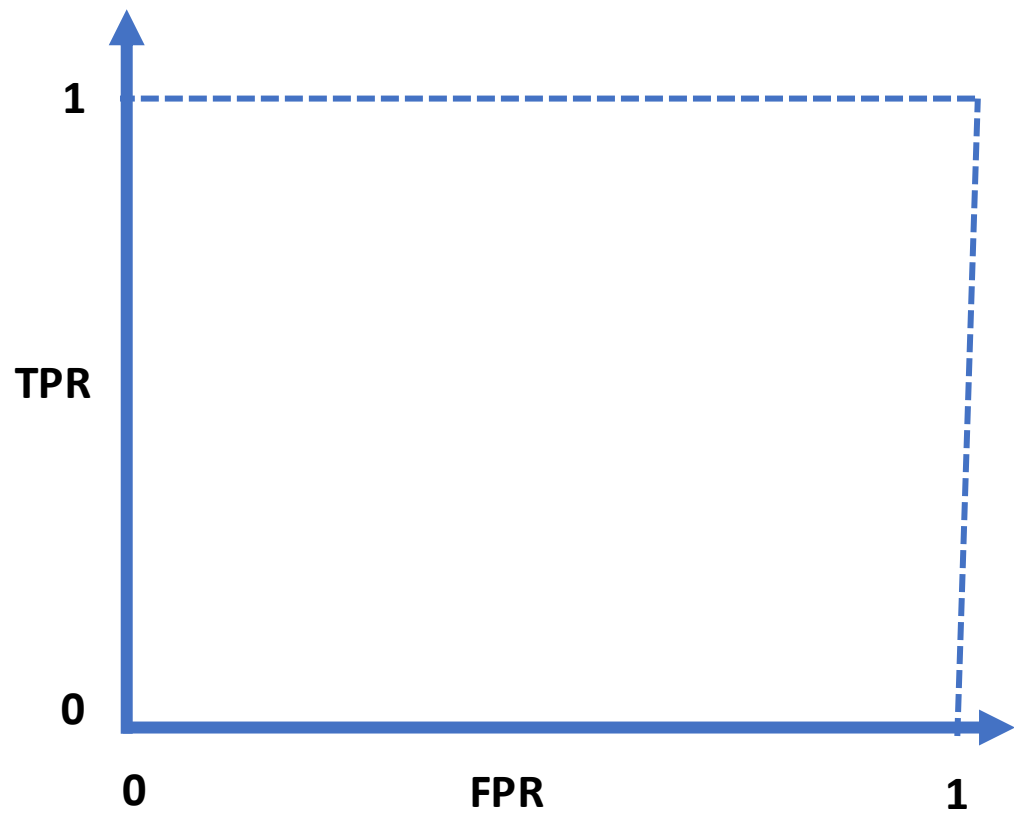
- A ROC (receiver operating characteristic curve) takes the threshold out of the process and is a graph showing the performance of a classification model at all classification thresholds. This curve plots two parameters:

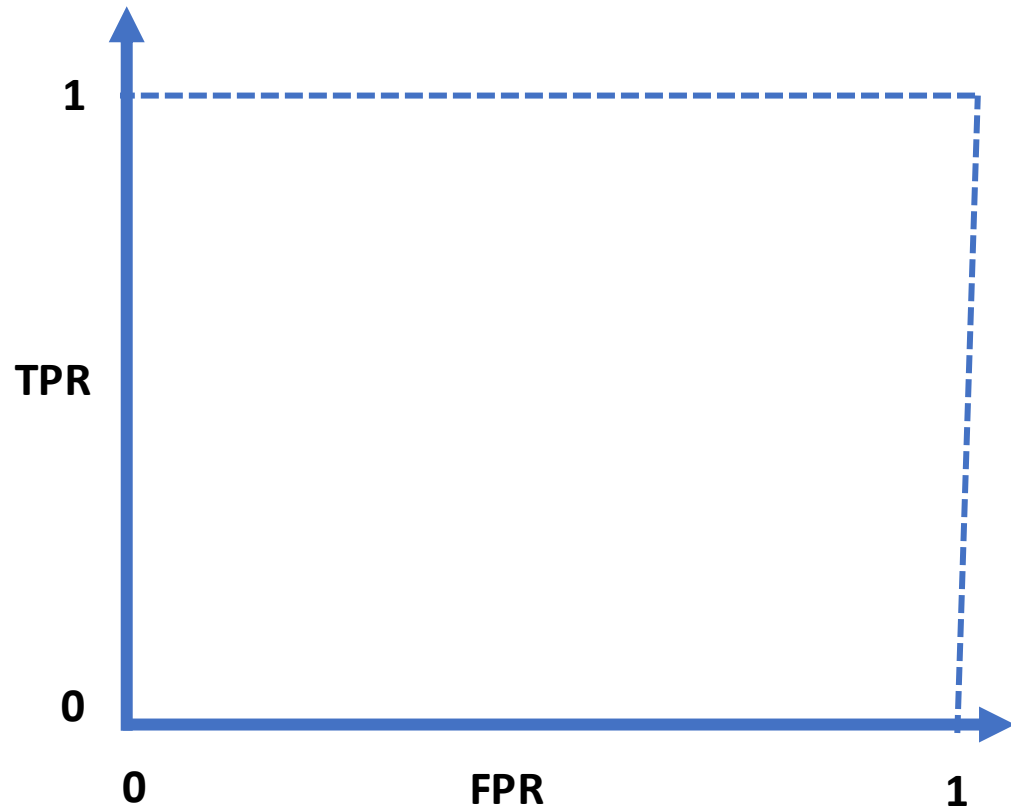
- True Positive Rate

$$TPR = \frac{TP}{TP + FN}$$

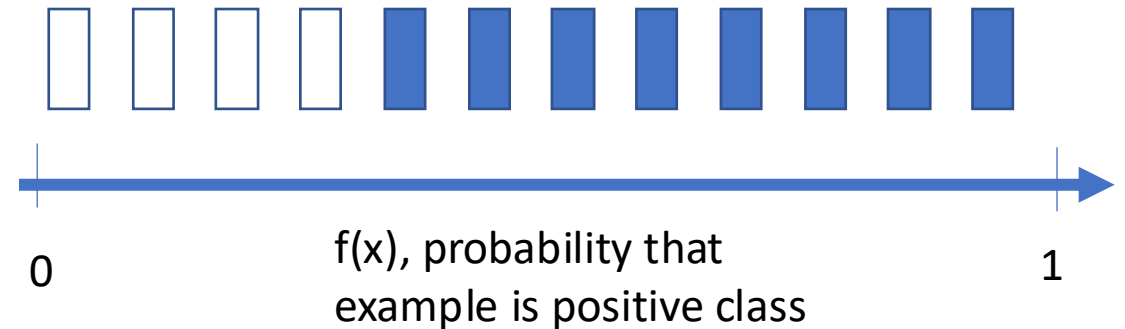
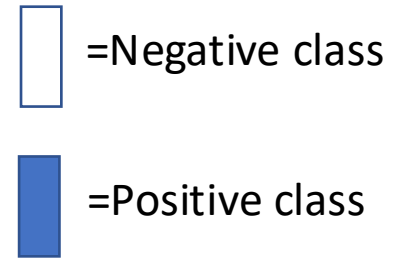
- False Positive Rate

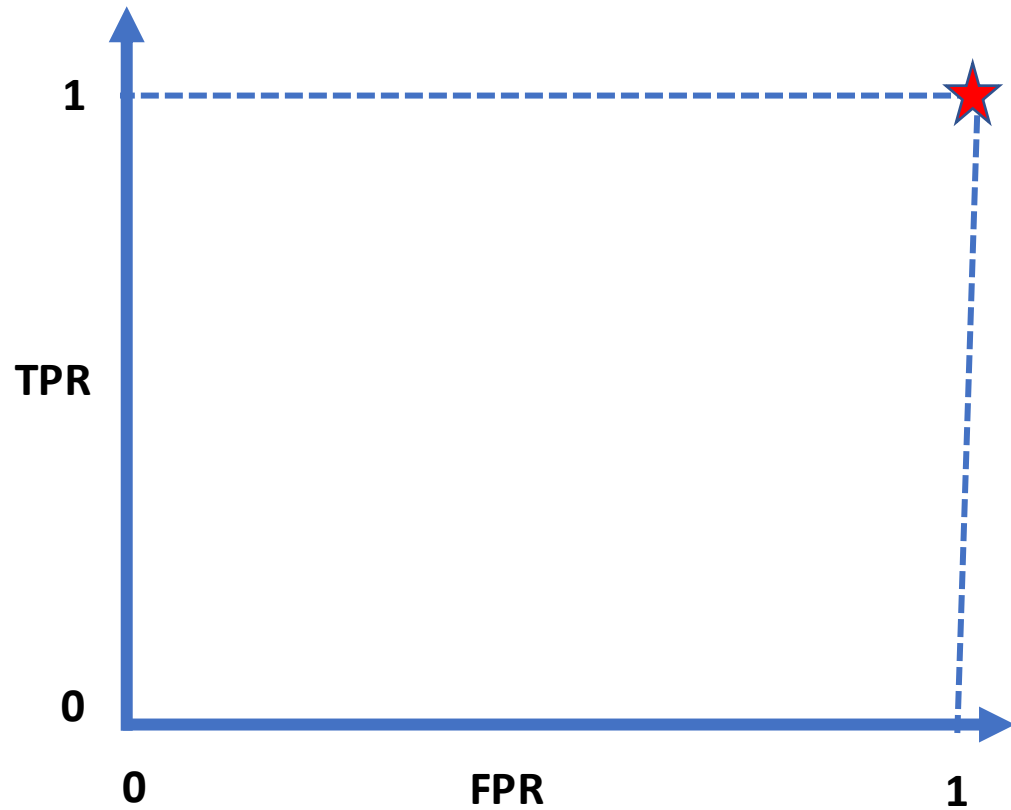
$$FPR = \frac{FP}{FP + TN}$$





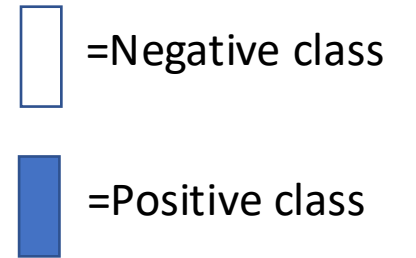
Lets say we 12 datapoints for testing.
And we arrange them according to the
probability that they are the positive class
according to our classifier



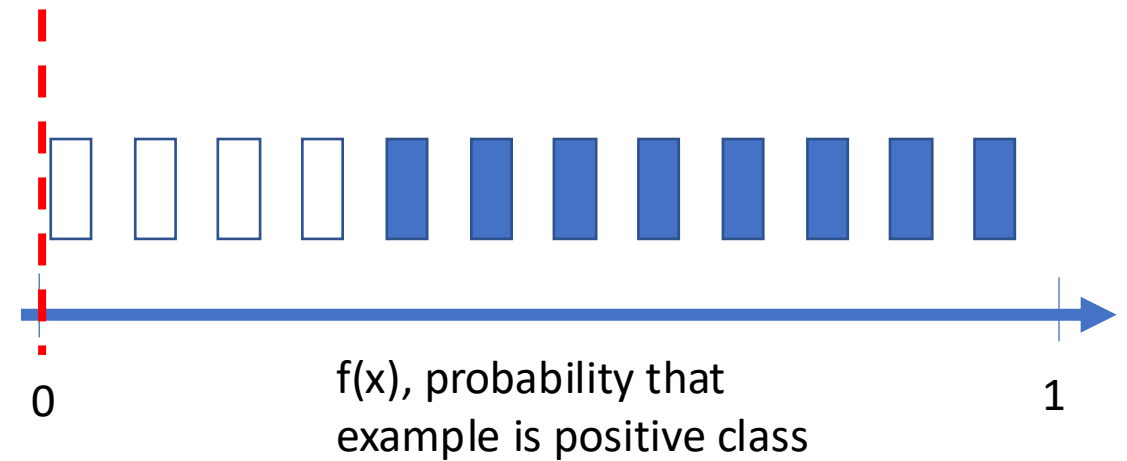


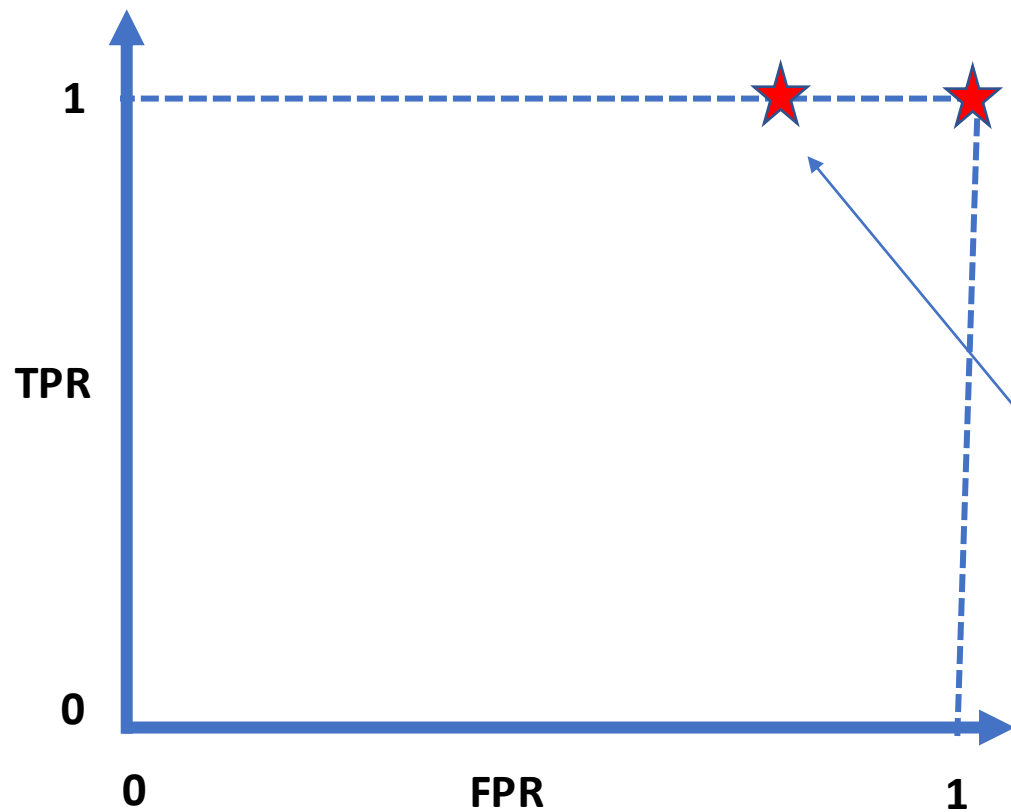
TPR=8/8
FPR=4/4

Lets say we 12 datapoints for testing.
And we arrange them according to the
probability that they are the positive class
according to our classifier



Lets set $t=0$



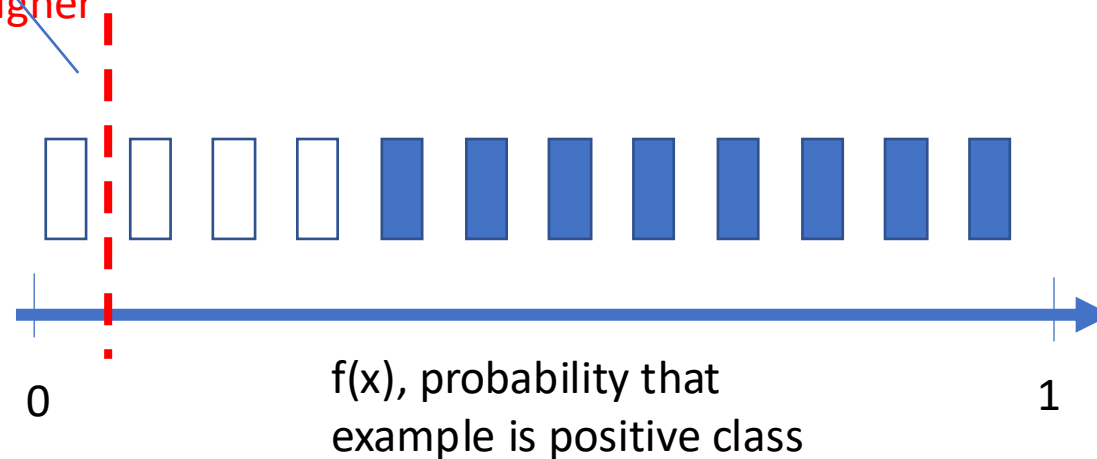


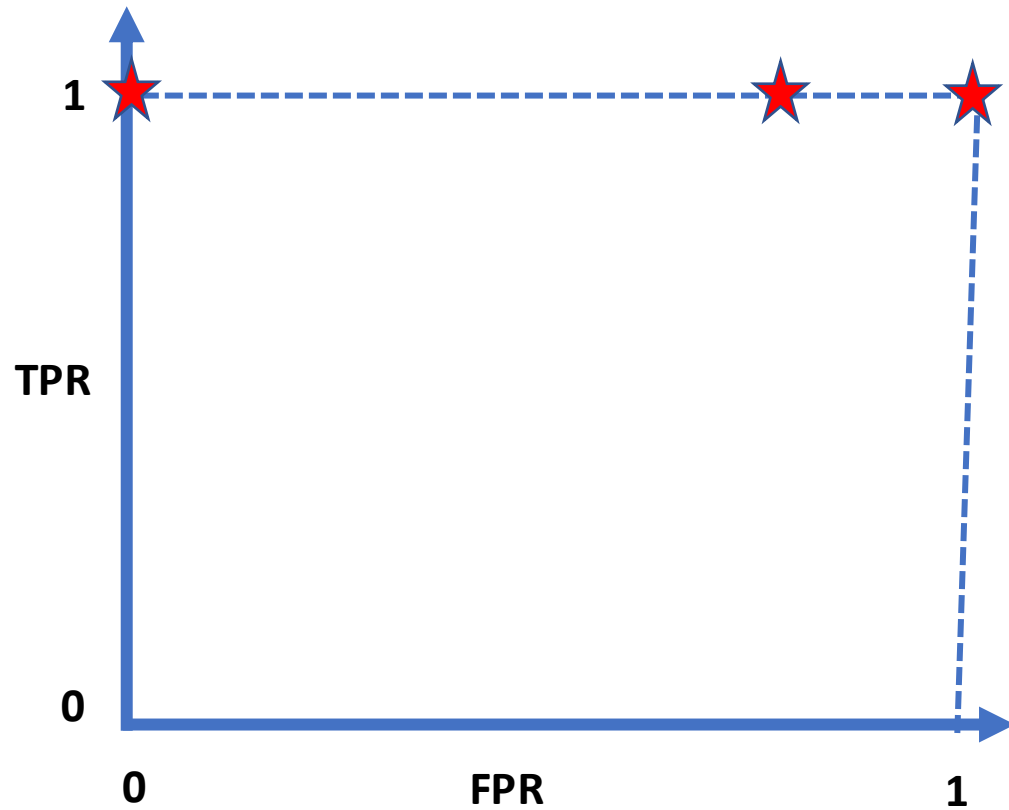
TPR=8/8
FPR=3/4

Lets say we 12 datapoints for testing.
And we arrange them according to the
probability that they are the positive class
according to our classifier

 =Negative class
 =Positive class

Lets set t a bit
higher



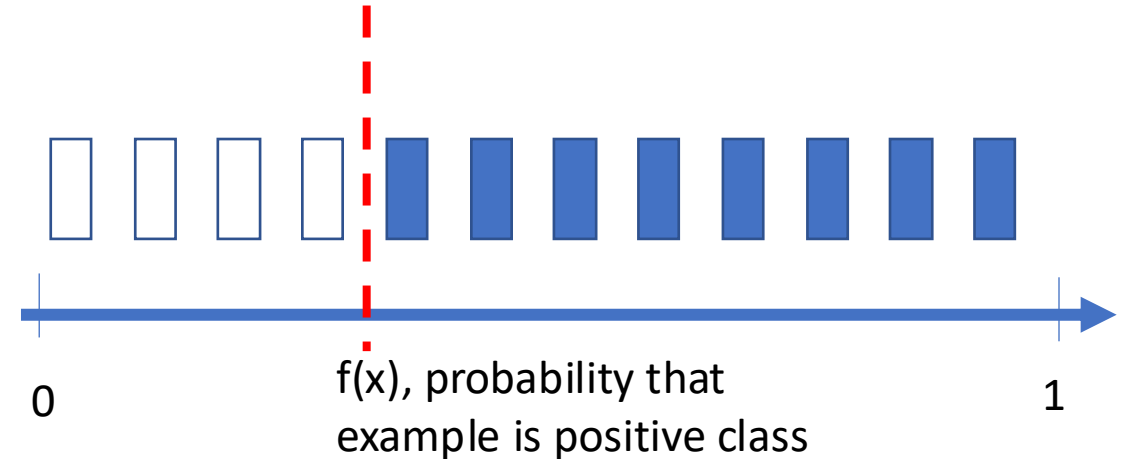


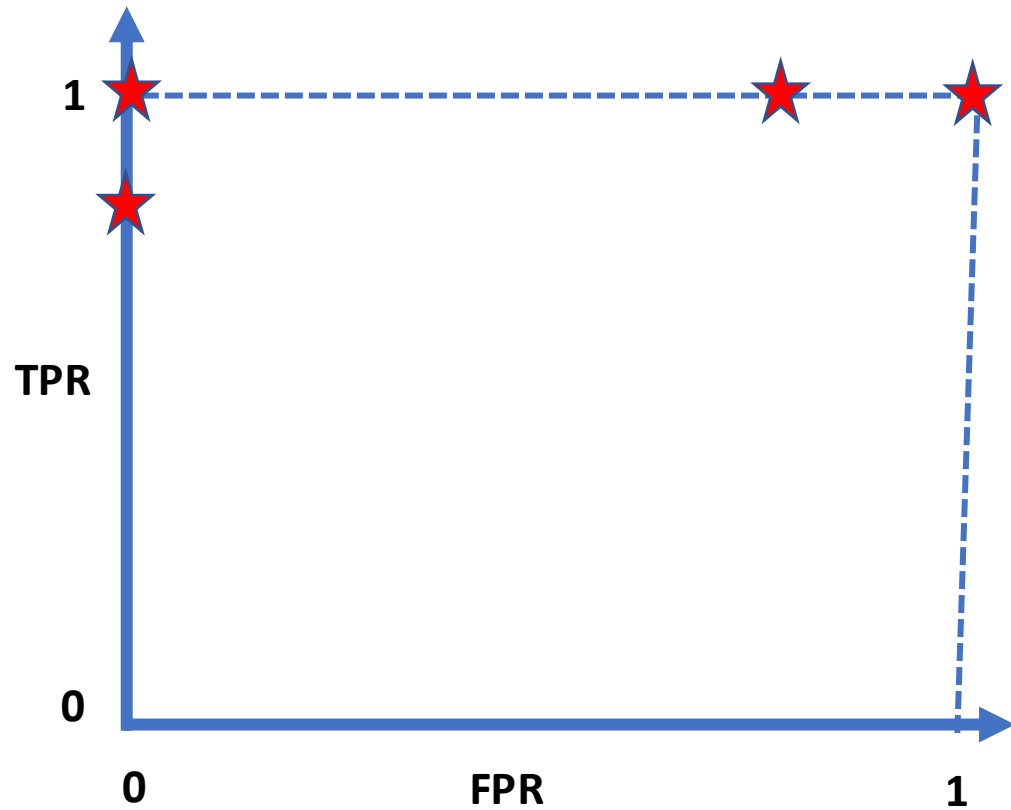
TPR=8/8
FPR=0

Lets say we 12 datapoints for testing.
And we arrange them according to the
probability that they are the positive class
according to our classifier

 =Negative class
 =Positive class

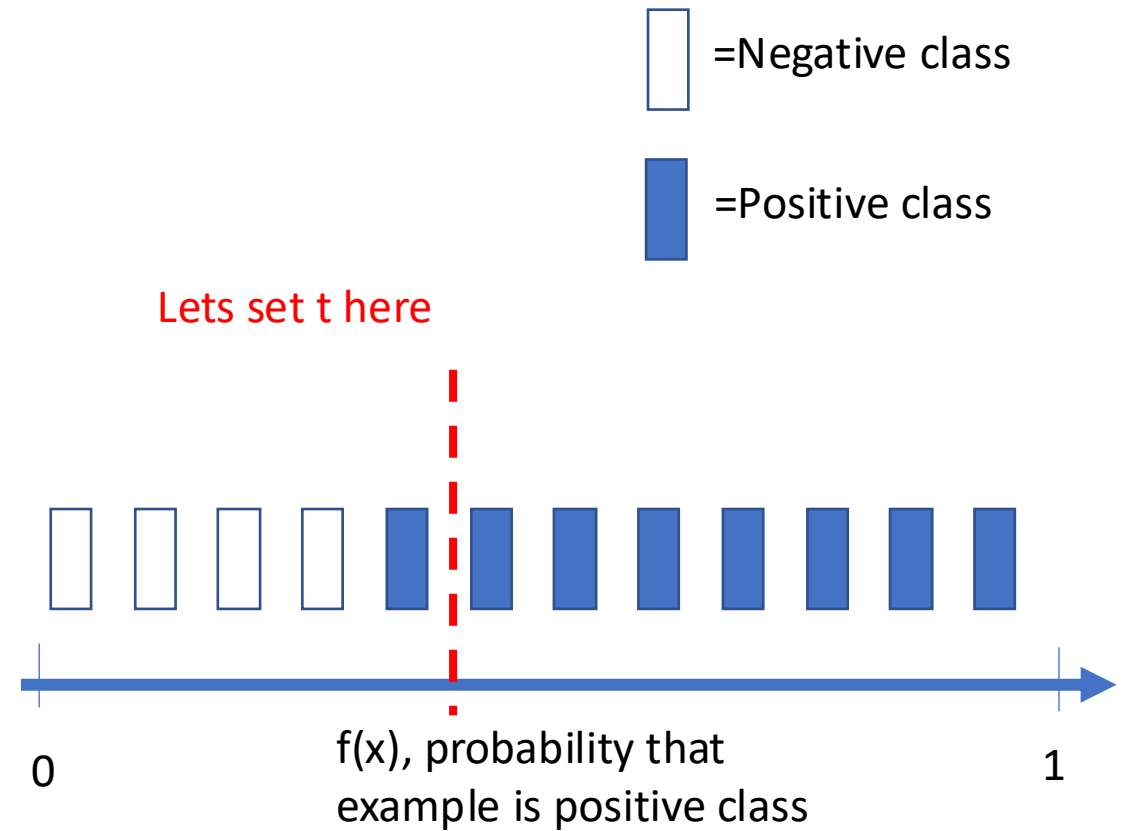
Lets set t just here

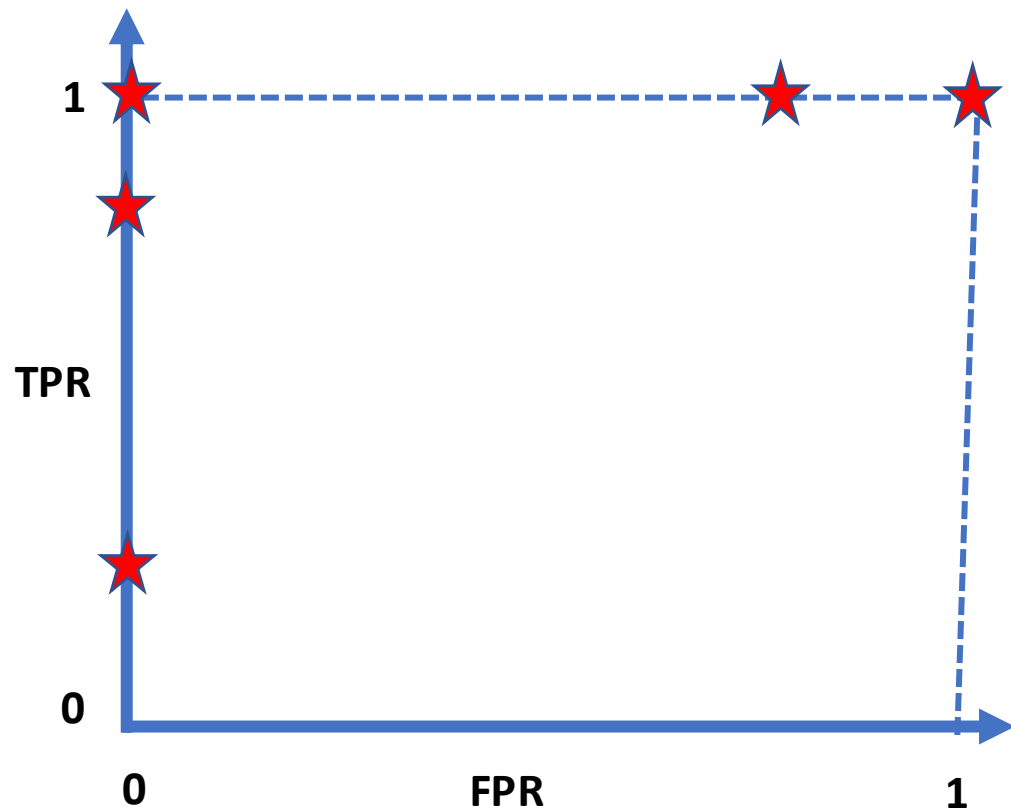




TPR=7/8
FPR=0

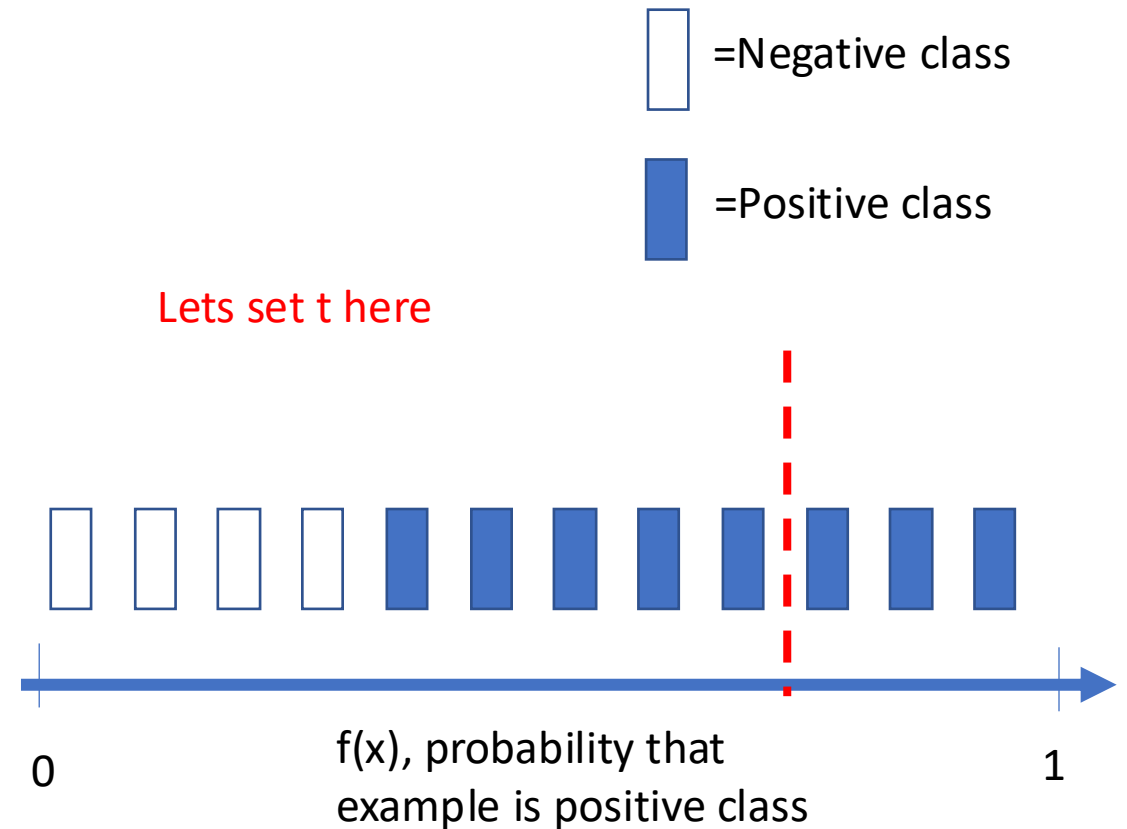
Lets say we 12 datapoints for testing.
And we arrange them according to the
probability that they are the positive class
according to our classifier

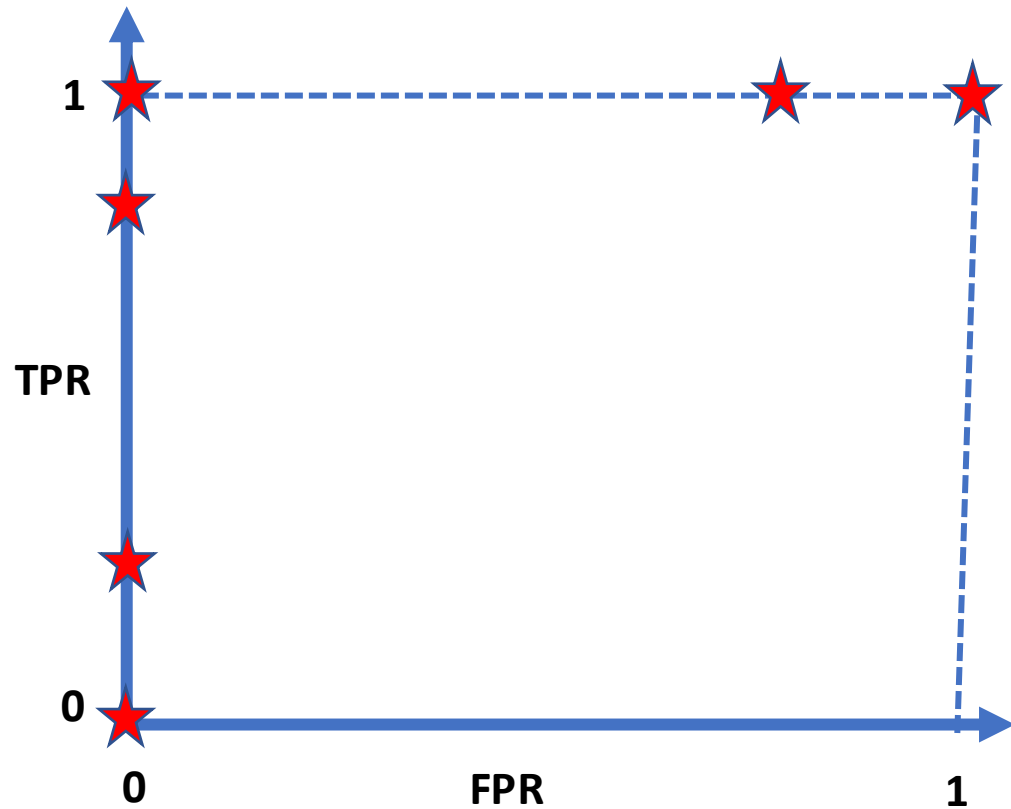




TPR=3/8
FPR=0

Lets say we 12 datapoints for testing.
And we arrange them according to the
probability that they are the positive class
according to our classifier



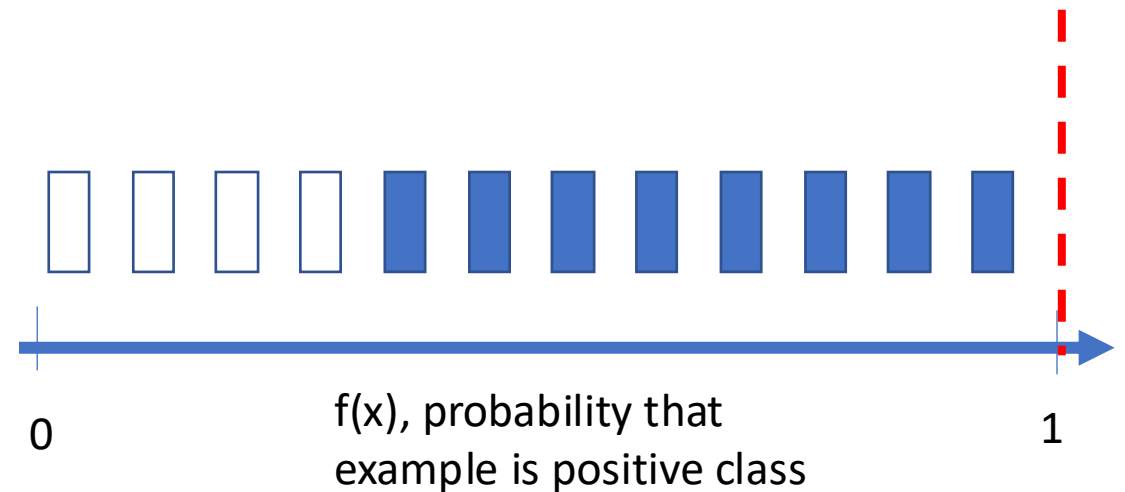


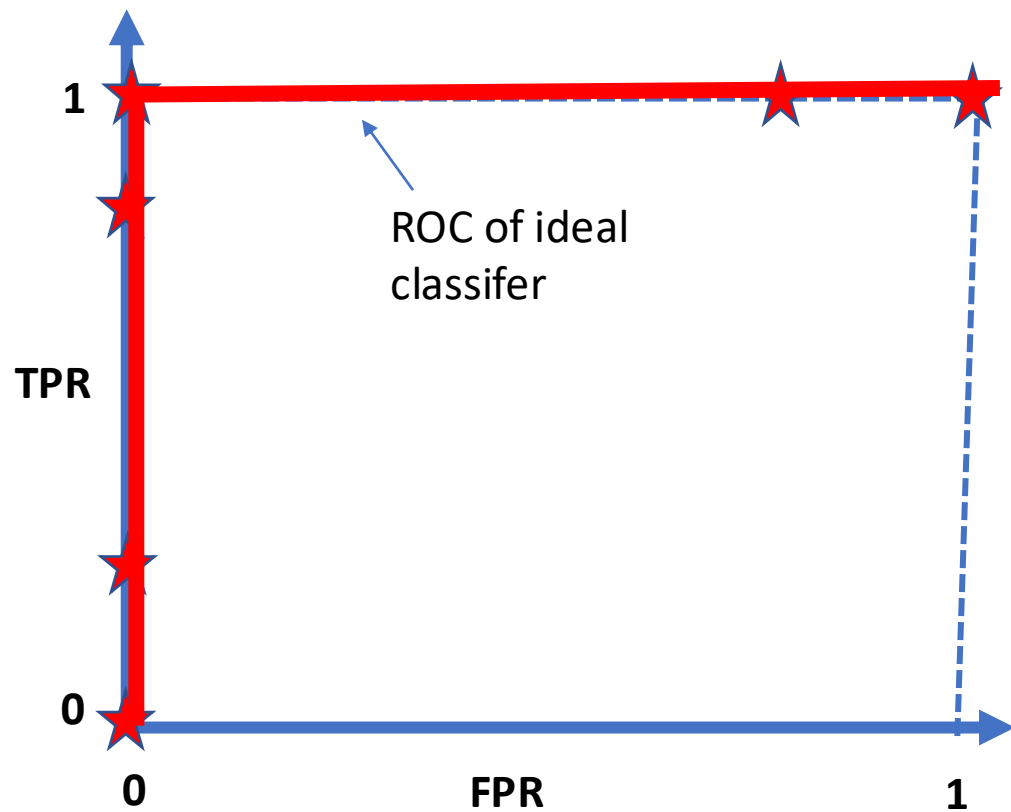
TPR=0
FPR=0

Lets say we 12 datapoints for testing.
And we arrange them according to the
probability that they are the positive class
according to our classifier

 =Negative class
 =Positive class

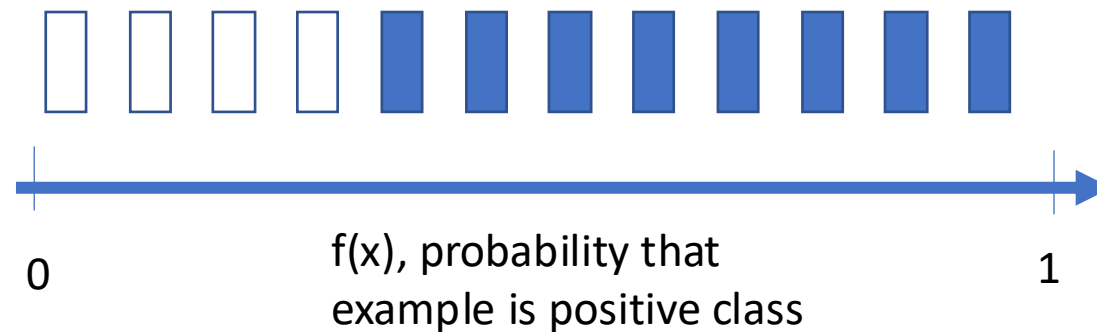
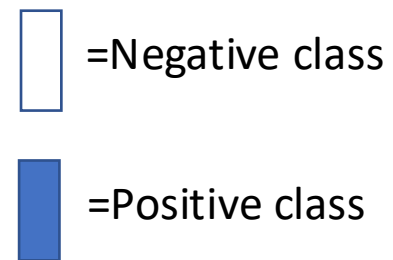
Lets set t here

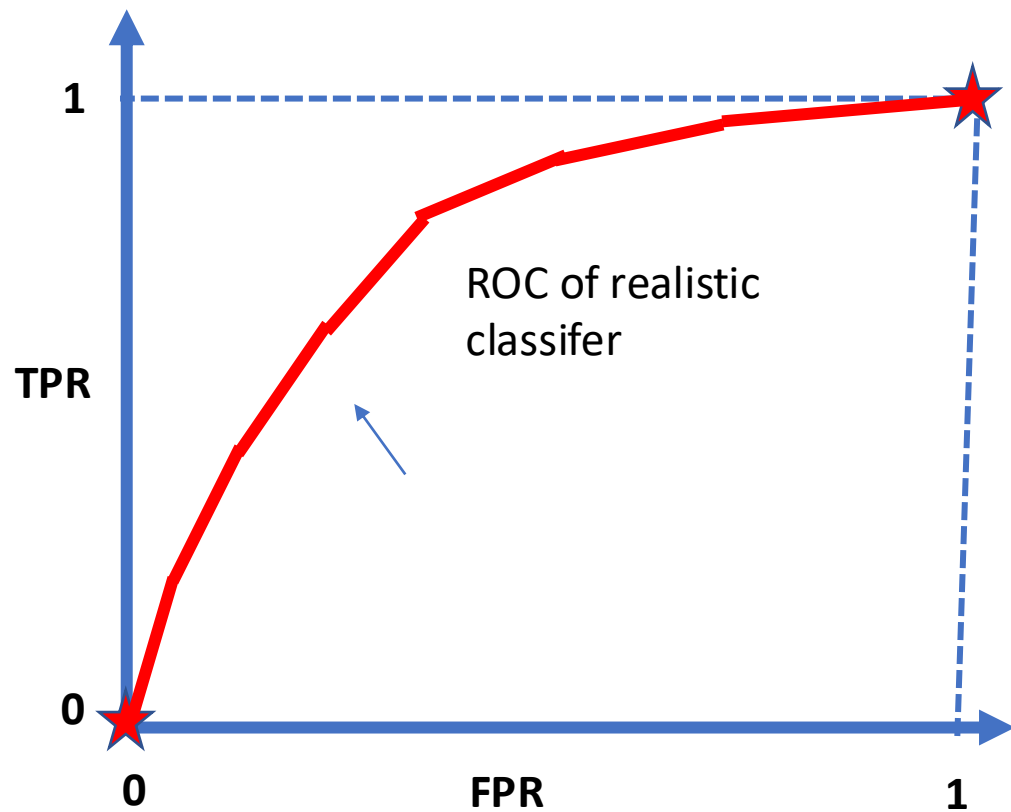




TPR=0
FPR=0

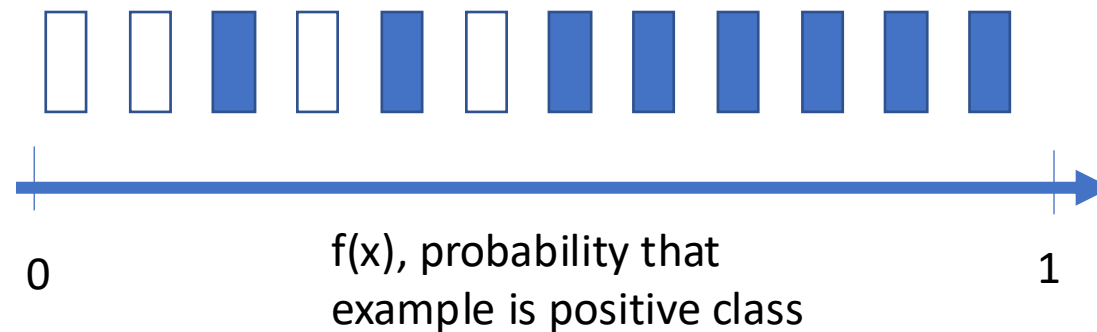
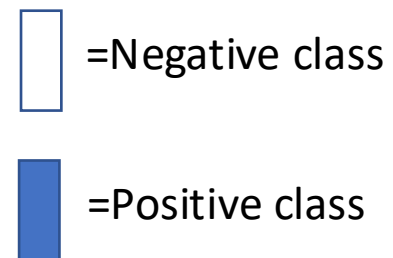
Lets say we 12 datapoints for testing.
And we arrange them according to the
probability that they are the positive class
according to our classifier





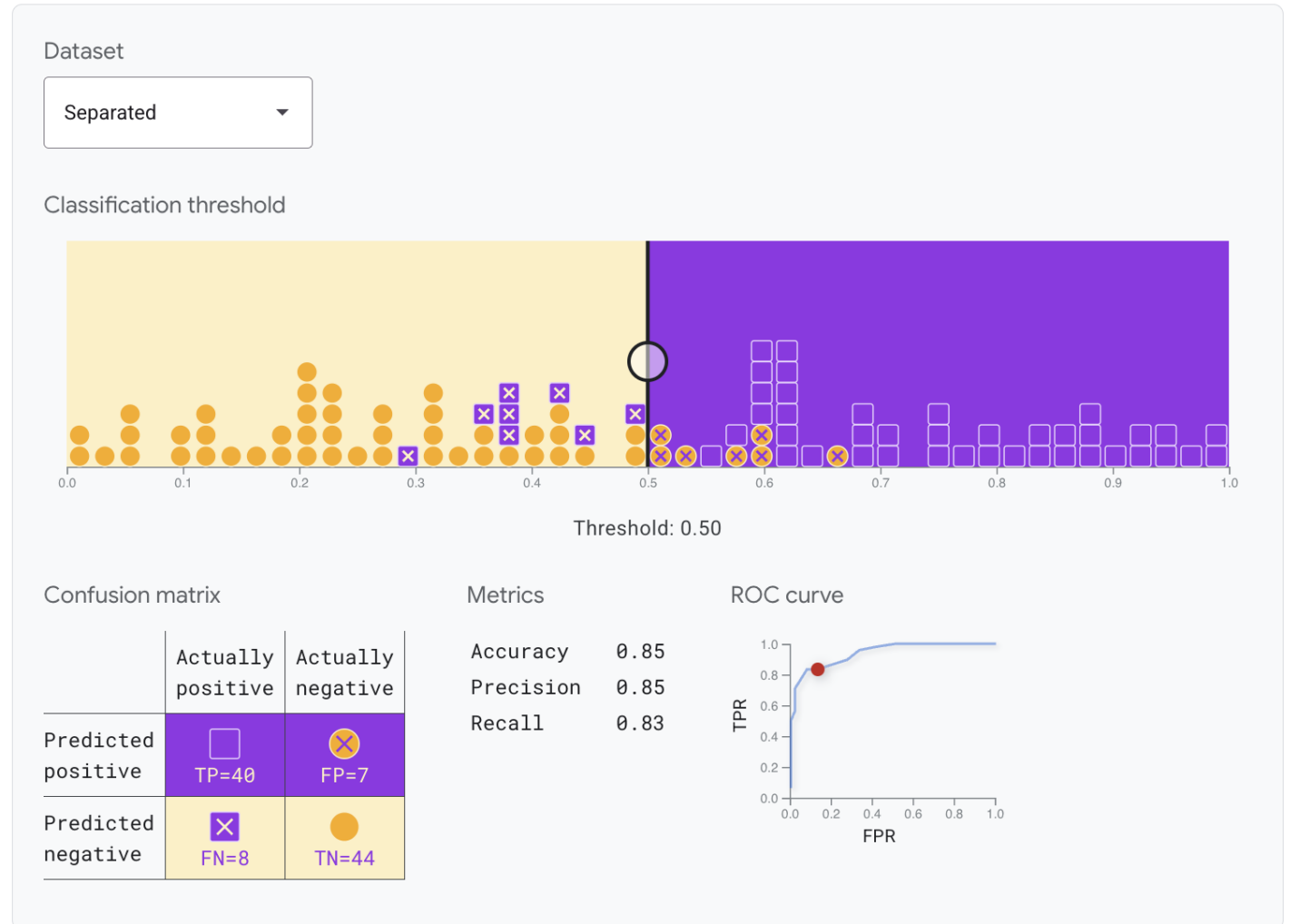
TPR=0
FPR=0

Lets say we 12 datapoints for testing.
And we arrange them according to the
probability that they are the positive class
according to our classifier



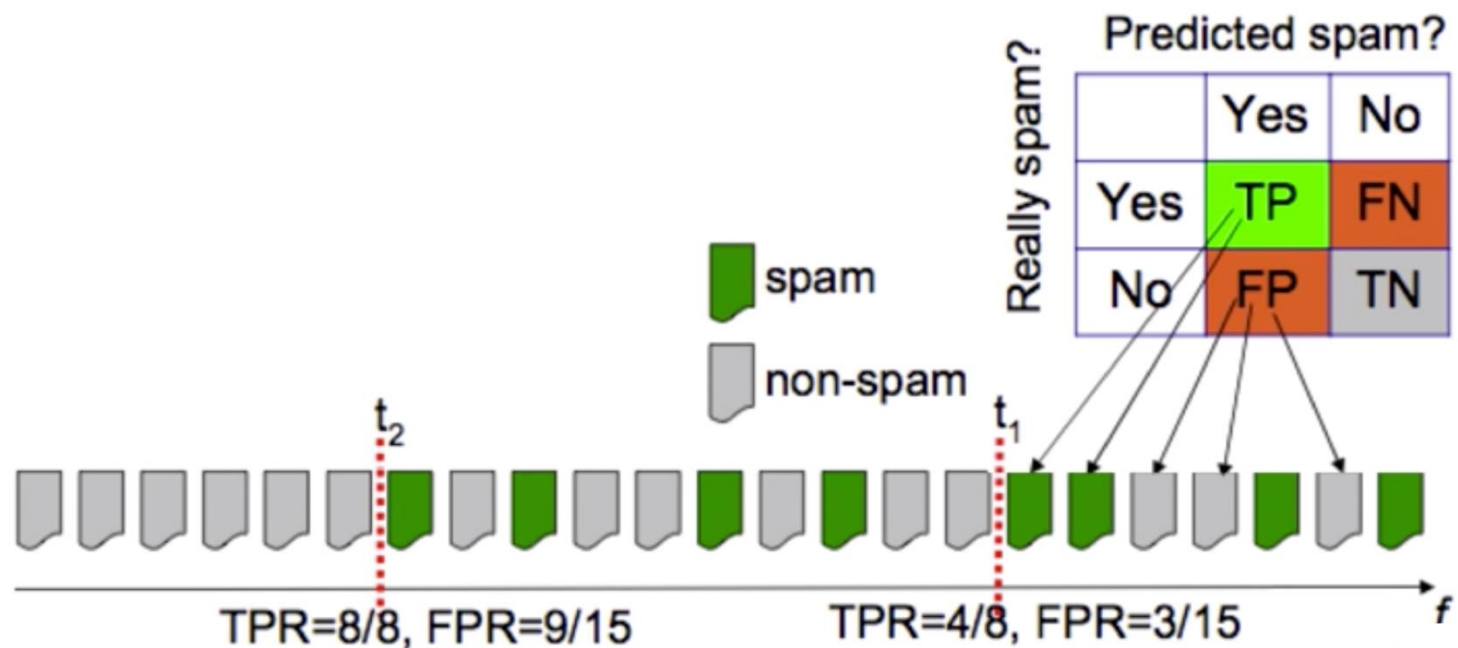
Interactive Demo - Google

- <https://developers.google.com/machine-learning/crash-course/classification/roc-and-auc>
- Please consider as part of the lecture notes



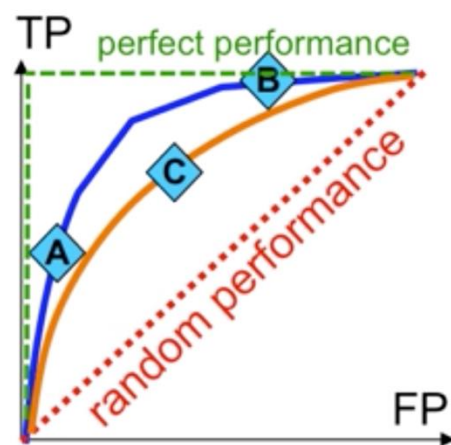
ROC Curves

- Many algorithms compute “confidence” $f(x)$
 - threshold to get decision: spam if $f(x) > t$, non-spam if $f(x) \leq t$
 - Naïve Bayes: $P(\text{spam}|x) > 0.5$, Linear/Logistic/SVM: $w^T x > 0$, Decision Tree: $p_+/p_- > 1$
 - threshold t determines error rates
 - False Positive rate = $P(f(x) > t | \text{ham})$, True Positive rate = $P(f(x) > t | \text{spam})$
- Receiver Operating Characteristic (ROC):



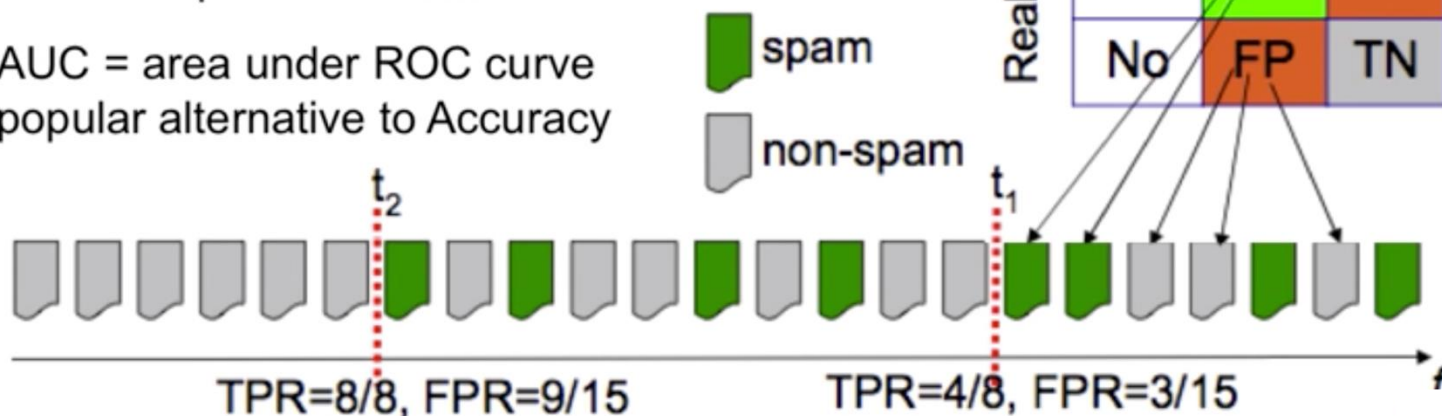
ROC Curves

- Many algorithms compute “confidence” $f(x)$
 - threshold to get decision: spam if $f(x) > t$, non-spam if $f(x) \leq t$
 - Naïve Bayes: $P(\text{spam}|x) > 0.5$, Linear/Logistic/SVM: $w^T x > 0$, Decision Tree: $p_+/p_- > 1$
 - threshold t determines error rates
 - False Positive rate = $P(f(x) > t | \text{ham})$, True Positive rate = $P(f(x) > t | \text{spam})$
- Receiver Operating Characteristic (ROC):
 - plot TPR vs. FPR as t varies from ∞ to $-\infty$



shows performance of system across all possible thresholds

AUC = area under ROC curve
popular alternative to Accuracy

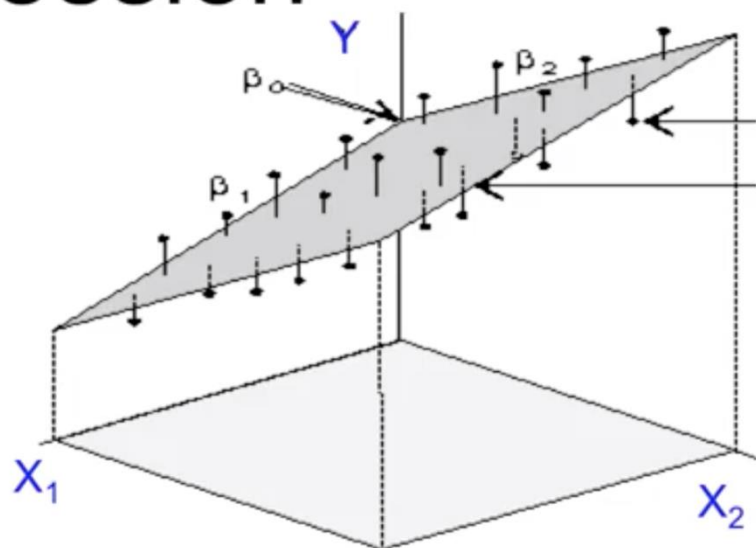


Miscellaneous ROC

- How do you calculate ROC for ensembles? Not on exam but in case you are curious check out this useful article - <https://community.wolfram.com/groups/-/m/t/944004>
- In general great notes on precision and recall and ROC - <https://willkoehrsen.github.io/statistics/learning/beyond-accuracy-precision-and-recall/>

Evaluating regression

- Classification:
 - count how often we are wrong
- Regression:
 - predict numbers y_i from inputs x_i
 - always wrong, but by how much?
 - distance between predicted & true values
 - (root) mean squared error:
 - popular, well-understood, nicely differentiable
 - sensitive to single large errors (outliers)
 - mean absolute error:
 - less sensitive to outliers
 - correlation coefficient
 - insensitive to mean & scale



$$\sqrt{\frac{1}{n} \sum_{i=1}^n \underbrace{(f(x_i))}_{\text{predicted}} - \underbrace{y_i}_{\text{true}})^2}$$

testing set

$$\frac{n \sum_{i=1}^n (f(x_i) - \mu_f)(y_i - \mu_y)}{\sqrt{\sum_{i=1}^n (f(x_i) - \mu_f)^2 \cdot \sum_{i=1}^n (y_i - \mu_y)^2}}$$

Mean Squared Error

- Average (squared) deviation from truth $\sqrt{\frac{1}{n} \sum_{i=1}^n (f(x_i) - y_i)^2}$

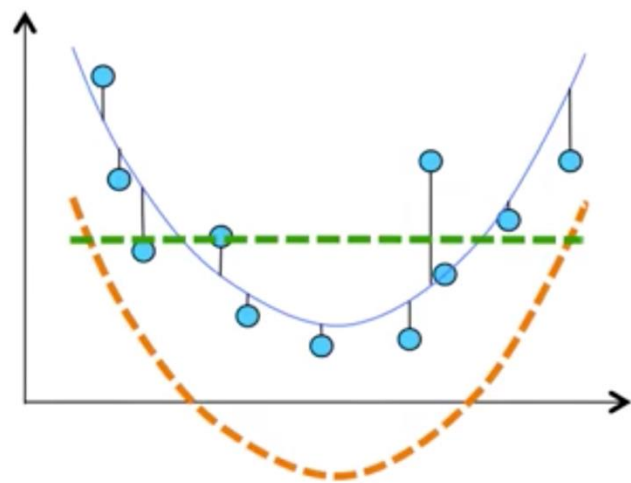
- Very sensitive to outliers

- 99 exact, 1 off by \$10
 - all 100 wrong by \$1
- } same MSE $\Rightarrow$ large effect on models

- Sensitive to mean / scale

- $\mu_y = 1/n \sum y_i$... good baseline

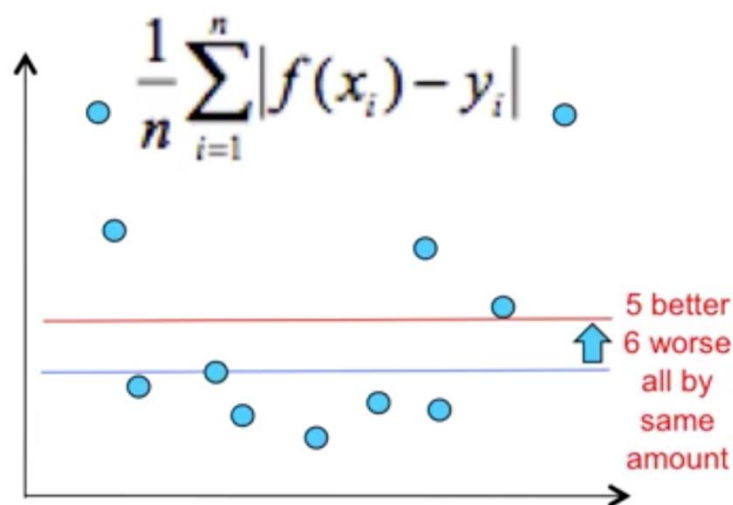
- Relative squared error (Weka)



$$\sqrt{\frac{\sum_{i=1}^n (f(x_i) - y_i)^2}{n}} \quad \left. \vphantom{\sqrt{\frac{\sum_{i=1}^n (f(x_i) - y_i)^2}{n}}} \right\} \text{MSE of predictor}$$
$$\sqrt{\frac{\sum_{i=1}^n (\mu_y - y_i)^2}{n}} \quad \left. \vphantom{\sqrt{\frac{\sum_{i=1}^n (\mu_y - y_i)^2}{n}}} \right\} \text{MSE when using the mean as a predictor}$$

Mean Absolute Error

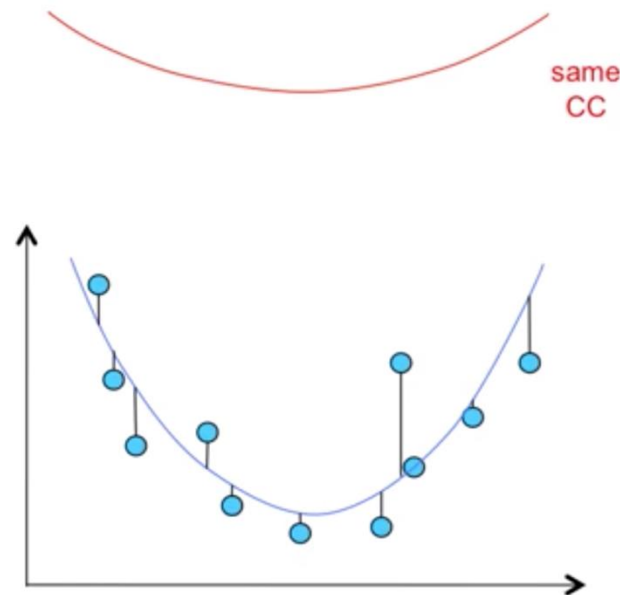
- Mean Absolute Error (MAE):
 - less sensitive to outliers
 - many small errors = one large error
 - best 0th order baseline: $\text{median}\{y_i\}$
 - not the mean as for MSE
- Median Absolute Deviation (MAD): $\text{med}\{|f(x_i) - y_i|\}$
 - robust, completely ignores outliers
 - can define similar squared error: $\text{median}\{(f(x_i) - y_i)^2\}$
 - difficult to work with (can't take derivatives)
- Sensitive to mean, scale



Correlation Coefficient

- Insensitive to mean / scale:

$$\frac{n \sum_{i=1}^n (f(x_i) - \mu_f)(y_i - \mu_y)}{\sqrt{\sum_{i=1}^n (f(x_i) - \mu_f)^2 \cdot \sum_{i=1}^n (y_i - \mu_y)^2}} = n \sum_{i=1}^n \underbrace{\frac{f(x_i) - \mu_f}{\sigma_f}}_{\text{prediction relative to mean}} \cdot \underbrace{\frac{y_i - \mu_y}{\sigma_y}}_{\text{truth relative to mean}}$$

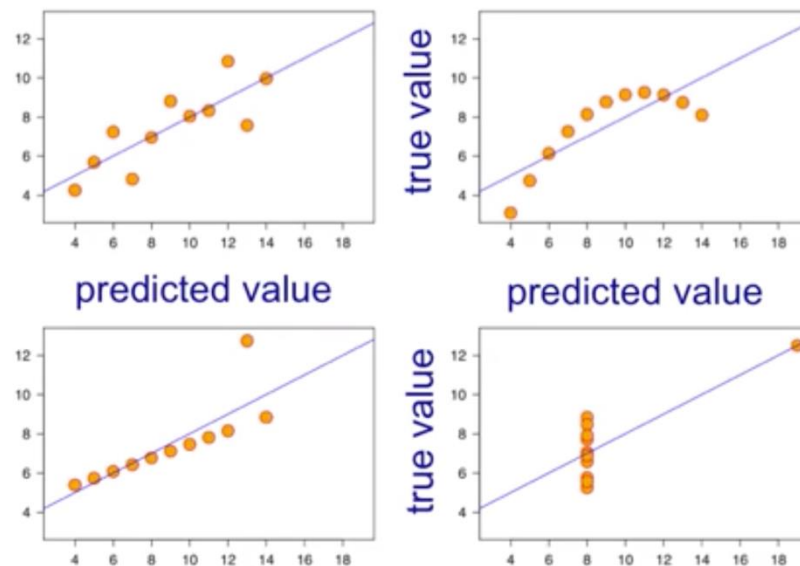


- Intuition: did you capture the relative ordering?

- output larger $f(x_i)$ for larger y_i
- output smaller $f(x_i)$ for smaller y_i
- useful for ranking tasks:
 - e.g. recommend a movie to a user

- Important to visualize data

- same CC for 4 predictors →



Evaluation – How to tell if your machine learning model is any good

- So after seeing all those measures – how do we use these to tell if our *model skill* is any good?
- For example we might achieve an F1 score of 0.6 in some particular machine learning task. Your team might ask you, “great work Jane, but is that good? It sort of sounds good but can you give us more information?”
- The next few slides deal with these awkward questions 😊

Model Skill Context

- Given the data you are using, your choice of tools, your computing resources, your stopping criteria, your implementation abilities it is not really possible to judge your achieved scores (machine learning skill) in **absolute terms**.
- It's like going for a run. While your own performance on a run can improve with training, if you really want to know if your runs are any good....what do you do? You compare your performance with other people.
- Similarly in machine learning – what we need to do is with the **same** data train a competing machine learning model. We measure performance then in **relative terms**.

Baseline Model Skill

- Obviously, like our running analogy, you need to compare with well accepted norms
- We call these a **baseline model**.
- A baseline model should be some well understood, straightforward and known predictive modelling algorithm.
 - Sometimes if it was a research paper, and you are trying to achieve the state of the art, you will find the “champion” method out there and use that. Kaggle competitions and so on reflect that approach
 - Sometimes you will pick a robust well known approach such as XGBoost, Random Forest, SVM, etc
 - For imbalanced classes some people use a majority class prediction as a baseline (more generally using the “mode” for a classification problem). Others, might use a random guess as a prediction.
 - Which ever you choose these are your baseline and your comparison should be measured against that.
 - If you were explaining the skill over and above baseline for non-technical users you might express it in terms of improvement in a business process. For example, in a quality classification task, you might say that “we identify 20% more failed samples than our existing approach and this saves us €10,000 in returns per week. ”

Evaluation: Evaluating Model Skill More Convincingly

Generalization

- We are now concerned with evaluation of our machine learning models. Generalization captures arguably the most important component of performance, which is the ability of the estimator to perform well on data it has not seen before. Lets think about this. We begin with our training and testing data.
- When we develop a machine learning algorithm to perform some task we rely on available data to train the model. For this we use training data.
- Training data $\{x^j, y^j\}$
 - These are examples that we used to train our predictor
 - E.g. Instagram images which have been labelled as cats or dogs

Generalization

However it is important to focus on the obvious point that we are building our machine learning model to perform its task on data it will encounter in the future. For future data, in the example of classification, we will not know the true labels which is what we want to predict. We will want to predict the labels as well as possible but we will be working with examples that our classifier has never seen before.

This is:

Future data: $\{x^i, ?\}$

- Examples that our classifier has never seen before
- E.g. images we will see in our Instagram feed tomorrow

Generalization

- Want to do well on future data, not training
 - Not very useful, we already know y^j
 - Easy to be perfect on training data (DT, kNN)
 - Does not mean you will do well on future data
 - Can overfit to idiosyncrasies of our training data

Under- and Over-fitting

- Over-fitting
 - Predictor too complex (flexible)
 - Fits “noise” in the training data
 - Patterns that will not reappear
- In other words when overfitting our model is learning noise or idiosyncrasies in our training data that will not appear in future data.

Under- and Over-fitting

- Over-fitting
 - Predictor too complex (flexible)
 - Fits “noise” in the training data
 - Patterns that will not reappear
 - More formally our estimator Q overfits the data if:
 - We can find another predictor Q' which makes more mistakes on training data $E_{\text{train}}(Q') > E_{\text{train}}(Q)$ but fewer mistakes on unseen data: $E_{\text{unseen}}(Q) > E_{\text{unseen}}(Q')$

Under- and Over-fitting

- Over-fitting
 - Predictor too complex (flexible)
 - Fits “noise” in the training data
 - Patterns that will not reappear
 - More formally our estimator Q overfits the data if:
 - We can find another predictor Q' which makes more mistakes on training data $E_{\text{train}}(Q') > E_{\text{train}}(Q)$ but fewer mistakes on unseen data: $E_{\text{unseen}}(Q') < E_{\text{unseen}}(Q)$
- Under-fitting:
 - In the case of underfitting our estimator is too simplistic (too rigid)
 - Not powerful enough to capture salient values in data
 - Can find another predictor Q' with smaller E_{train} and E_{unseen}

Under- and Over-fitting

- Over-fitting

- Predictor too complex (flexible)

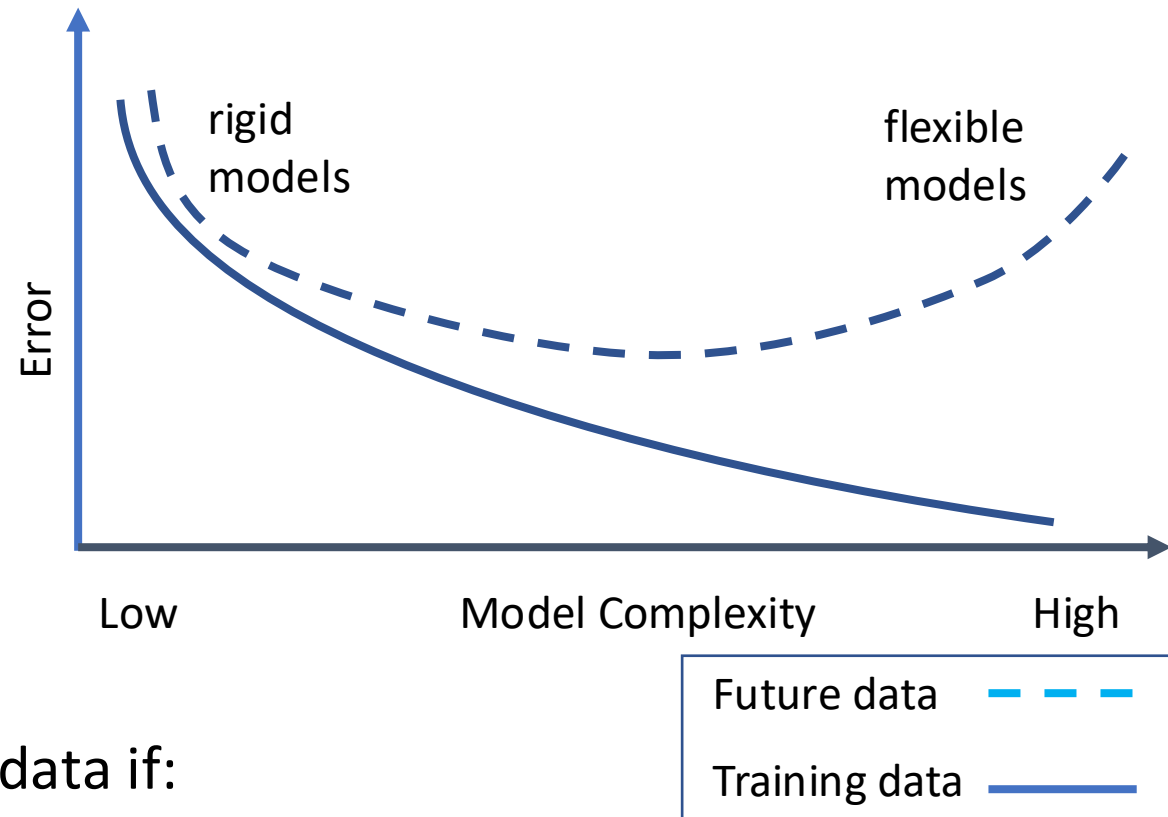
- Fits “noise” in the training data
 - Patterns that will not reappear

- More formally our estimator Q overfits the data if:

- We can find another predictor Q' which makes more mistakes on training data $E_{\text{train}}(Q') > E_{\text{train}}(Q)$ but fewer mistakes on unseen data: $E_{\text{unseen}}(Q) > E_{\text{unseen}}(Q')$

- Under-fitting:

- In the case of underfitting our estimator is too simplistic (too rigid)
 - Not powerful enough to capture salient values in data
 - Can find another predictor Q' with smaller E_{train} and E_{unseen}



Under- and Over-fitting

- Over-fitting

- Predictor too complex (flexible)

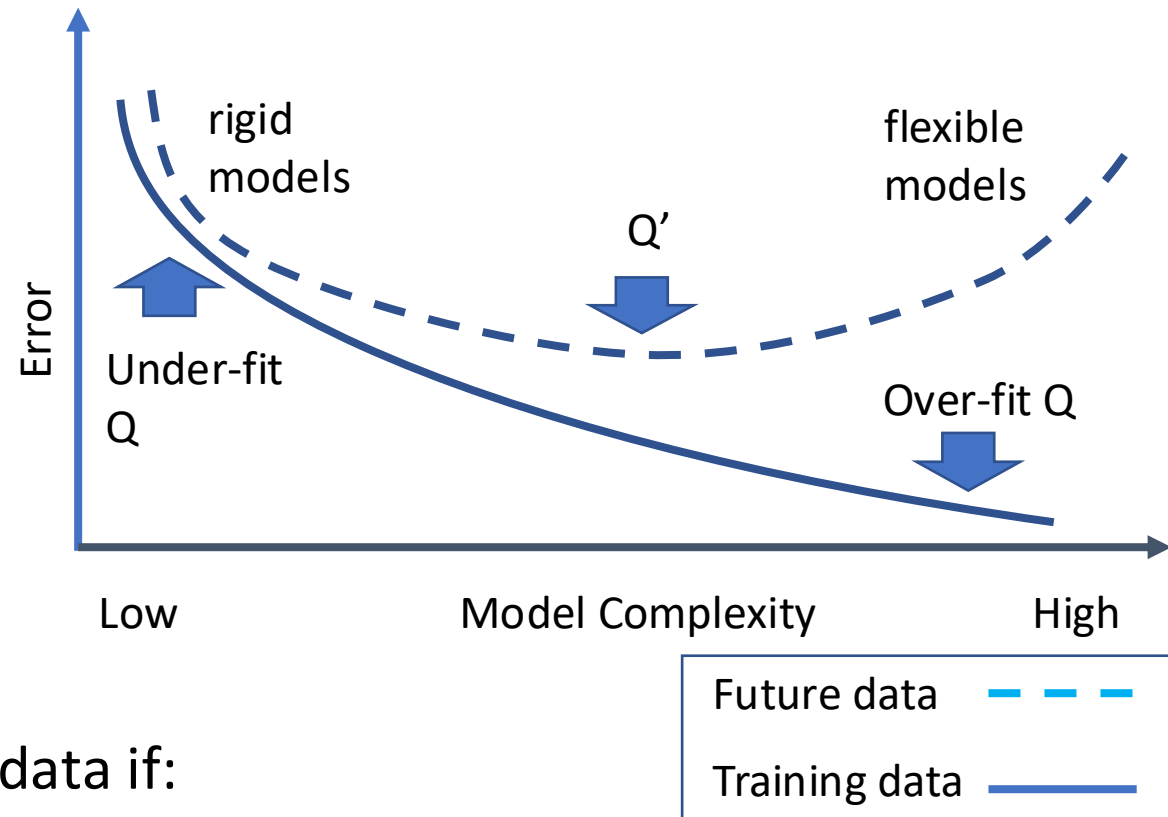
- Fits “noise” in the training data
 - Patterns that will not reappear

- More formally our estimator Q overfits the data if:

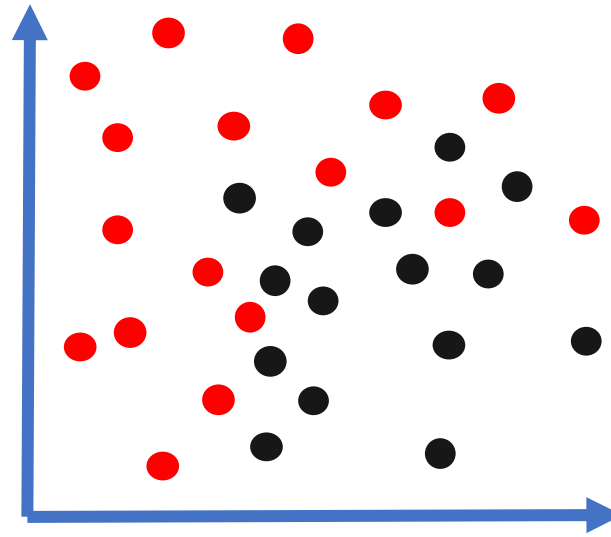
- We can find another predictor Q' which makes more mistakes on training data $E_{\text{train}}(Q') > E_{\text{train}}(Q)$ but fewer mistakes on unseen data: $E_{\text{unseen}}(Q') < E_{\text{unseen}}(Q)$

- Under-fitting:

- In the case of underfitting our estimator is too simplistic (too rigid)
 - Not powerful enough to capture salient values in data
 - Can find another predictor Q' with smaller E_{train} and E_{unseen}

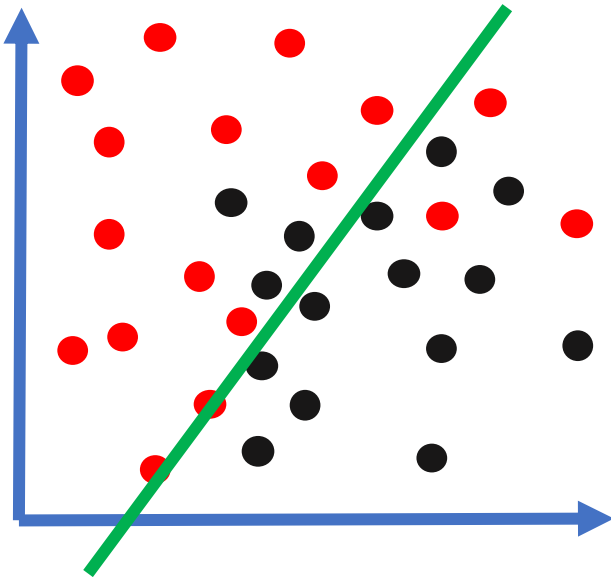


Examples of Under- and Over-fitting in Classification

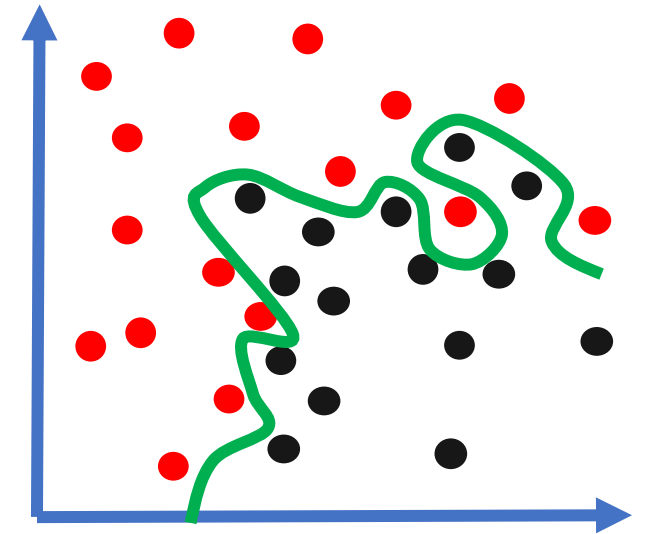


Good fit

Underfitting



Overfitting

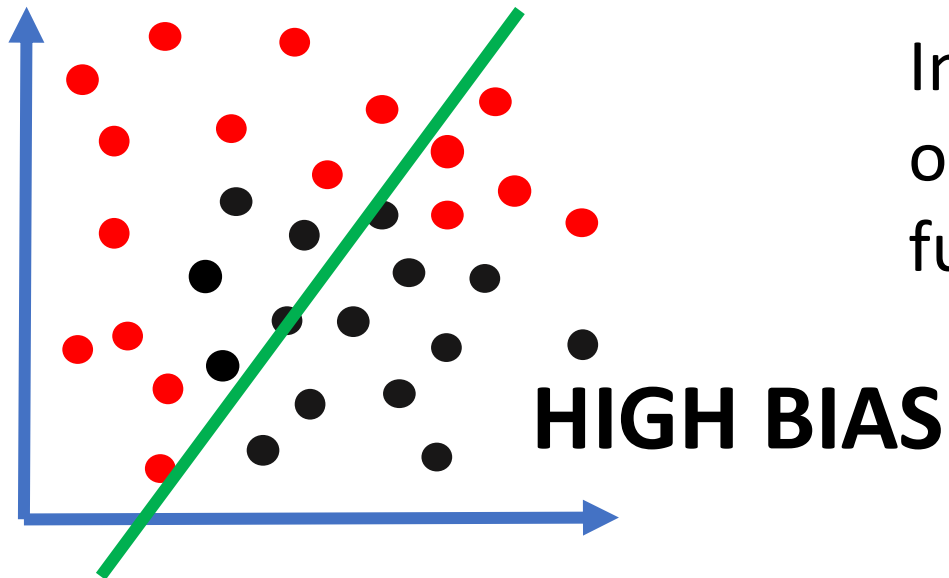


Overfitting vs Underfitting vs Bias vs Variance

- We often hear about “bias” vs “variance” in machine learning models and often in the same context as overfitting and underfitting. It is worth examining these concepts a little further with the objective of understanding the distinctions involved.
- As we have overfitting refers to a situation where the model performs very well on the data it was trained on but performs poorly when confronted with unseen data, i.e. data it has not been trained on or poor generalization.
- Underfitting refers to a situation where the model does not perform well even with the data it has been trained on. Also will perform poorly with unseen data.
- We can described both underfitting and overfitting in terms of bias and variance

Bias and Variance in Machine Learning

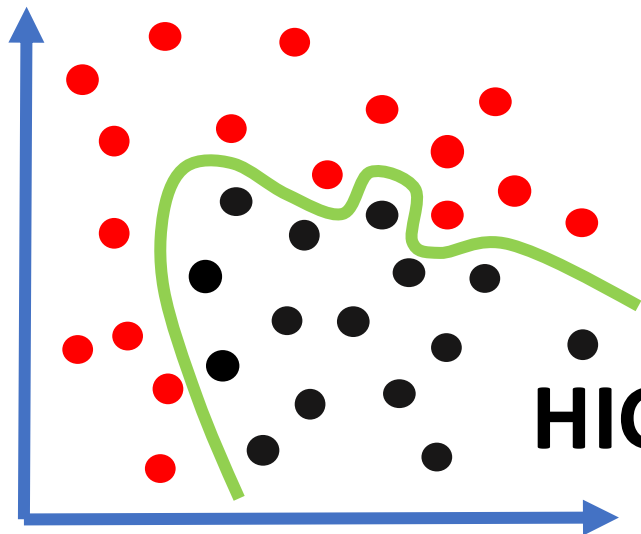
- A model is said to have **high bias** when the very structure of the model does not reflect the structure inherent in the data. The figure below is such an example where we have a linear structure to our model but the required decision boundary suggests a much more non linear model is necessary



In this case, the model will perform poorly on the training data and most likely any future data.

Bias and Variance in Machine Learning

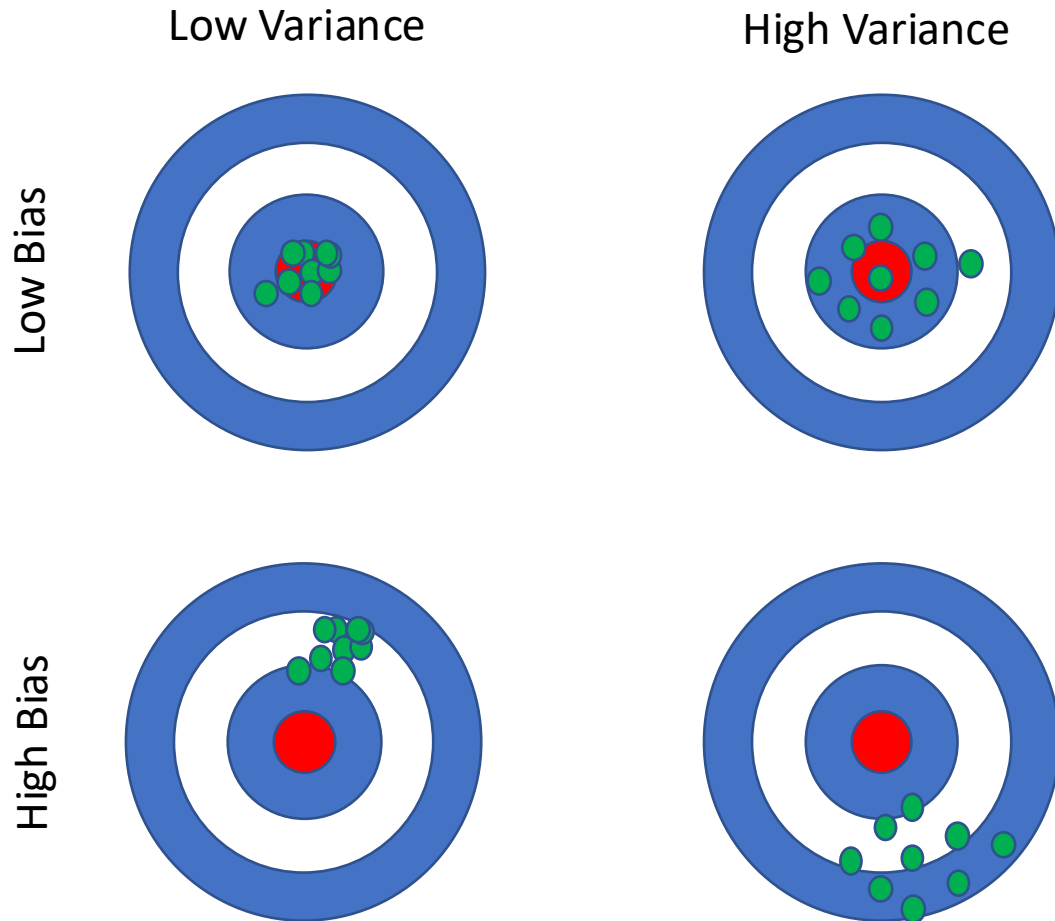
- A model is said to have **high variance** when it describes the specific training data very well. If you see that when trained on a different set of training data for the same problem that a very different model is produced then you know your model has high variance. **The amount of variance refers to how much the model changes with training sets.**



In this case, the model will perform poorly on unseen, future data as the model is too specific to the specific training set it has been trained on .

HIGH VARIANCE

The Classic Bias Variance Illustration



With this understanding of bias and variance we can see that our ideal model corresponds to low bias and low variance, i.e. top left corner in the image shown.

Low bias and high variance corresponds to an overfitted model as performance is sensitive to data trained with.

High bias and low variance is representative of underfitting.

Finally the bottom right is the opposite of our ideal model as it has worst performance on training and test data.

Generally the more complex a model the higher its variance and the lower its bias.

The opposite is true as models become less complex.

Flexible vs Inflexible predictors

- Each dataset needs different levels of “flexibility”
 - Depends on task complexity and available data
 - Need an adjustment setting to tune from rigid/flexible predictors
- Most learning algorithms have such knobs:
 - NB: attribute choices, .i.e how many,
 - DT: number of nodes, pruning strategy. We can go from shallow to deep trees.
 - kNN: choice of k
 - SVM: kernel type, C parameter, regularization
 - Regression order of the polynomial

2024 Slide - Grokking

- **Grokking** is a term that originated from the 1961 science fiction novel **Stranger in a Strange Land** by Robert A. Heinlein. In the novel, "grok" means to understand something so deeply and intuitively that you become one with it. In modern tech and AI communities, "grokking" has taken on a more specialized meaning.
- In the context of machine learning, ****grokking**** describes a phenomenon where a neural network initially struggles to learn a task (with poor performance during early training), but after extensive training on the data, it suddenly and dramatically improves its performance. This improvement happens after what would normally be considered "overtraining" (when the model has seen the data so many times that you would expect it to overfit).
- The term highlights an intriguing aspect of deep learning that is not yet fully understood: why and how, after a period of mediocre performance, the network suddenly figures out the task at hand. The neural network doesn't just memorize the training data, but rather "grokks" the underlying patterns or structure of the data.
- This phenomenon was first widely discussed in a paper by OpenAI in 2021, where researchers explored the behavior in the context of algorithmic tasks, showing that the neural network eventually "grokked" generalizable patterns rather than memorizing specific instances.
- In summary, in AI, ****grokking**** refers to a neural network's sudden realization of the structure of a problem, often after a period of overtraining, where its performance improves drastically without overfitting the training data.

Training vs Generalization Error

- Training error:
- Generalization error:
 - How well we will do on future data
 - Don't know what future data x^j will be
 - Don't know what labels y^j it will have

$$E_{train} = \frac{1}{M} \sum_{j=1}^M \text{Error}(Q_{\theta}(x^j), y^j)$$

Training examples

Value we predicted

True value

Training vs Generalization Error

- Training error:
- Generalization error E_{unseen} :
 - How well we will do on future data
 - Don't know what future data x^j will be
 - Don't know what labels y^j it will have
 - But know the "range" of all possible $\{x, y\}$
 - x : all possible 20x20 black/white bitmaps
 - y : $\{0, 1, \dots, 9\}$ (digits)

Can never compute
generalization error

$$E_{\text{train}} = \frac{1}{M} \sum_{j=1}^M \text{Error}(Q_{\theta}(x^j), y^j)$$

Training examples Value we predicted True value

Usually $E_{\text{train}} \leq E_{\text{unseen}}$

$$E_{\text{unseen}} = \int \underbrace{\text{error}(Q_{\theta}(x), y)}_{\text{Error as before}} \underbrace{p(y, x) dx}_{\text{How often we expect to see such } x \text{ and } y}$$

Sum over all possible x and y

Estimating Generalization Error

- Testing Error:
 - Set aside part of training data (testing set)

$$E_{train} = \frac{1}{M} \sum_{j=1}^M Error(Q_{\theta}(x^j), y^j)$$

Estimating Generalization Error

- Testing Error:
 - Set aside part of training data (testing set)
 - Set aside part of training data (testing set)
 - Learn a predictor without using any of this data

$$E_{train} = \frac{1}{M} \sum_{j=1}^M Error(Q_{\theta}(x^j), y^j)$$

Estimating Generalization Error

- Testing Error:

- Set aside part of training data (testing set)
 - Set aside part of training data (testing set)
 - Learn a predictor without using any of this data
 - Predict values for testing set, compute error
 - This will give an **estimate** of the true generalization error
 - If testing set is **unbiased** sample from $p(x,y)$: $\lim_{n \rightarrow \infty} E_{test} = E_{unseen}$
 - How close? Depends on n .

$$E_{train} = \frac{1}{M} \sum_{j=1}^M Error(Q_{\theta}(x^j), y^j)$$

Estimating Generalization Error

- Testing Error:

$$E_{train} = \frac{1}{M} \sum_{j=1}^M Error(Q_{\theta}(x^j), y^j)$$

- Set aside part of training data (testing set)
 - Set aside part of training data (testing set)
 - Learn a predictor without using any of this data
 - Predict values for testing set, compute error
 - This will give an **estimate** of the true generalization error
 - If testing set is **unbiased** sample from $p(x,y)$: $\lim_{n \rightarrow \infty} E_{test} = E_{gen}$
 - How close? Depends on n .

- Example: binary classification, 100 instances

- Assume: 75 classified correctly and 25 incorrectly
- $E_{test}=0.25$, E_{unseen} around 0.25, but how close?

Confidence Interval for Future Error

- What range of errors can we expect for **future** test sets.
 - $E_{test} \pm \Delta E$ such that 95% of future tests fall within that interval

Confidence Interval for Future Error

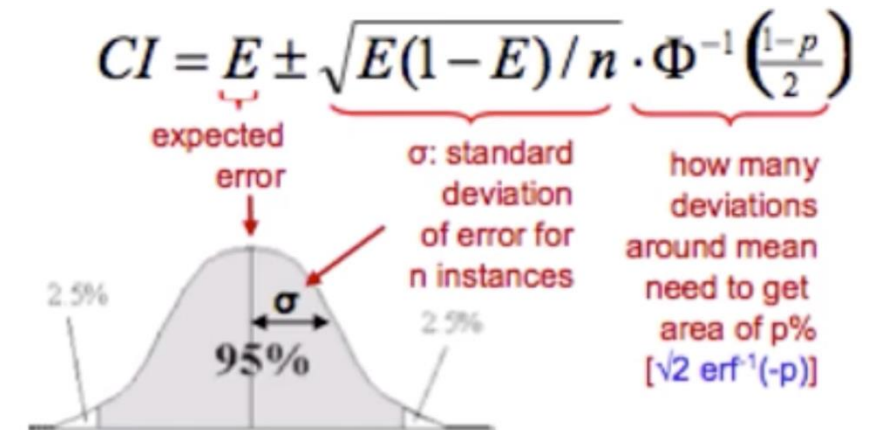
- What range of errors can we expect for **future** test sets.
 - $E_{test} \pm \Delta E$ such that 95% of future tests fall within that interval
- E_{test} is an unbiased estimate of $E = \textit{true error rate}$
 - $E =$ probability our system will misclassify a random instance

Confidence Interval for Future Error

- What range of errors can we expect for **future** test sets.
 - $E_{test} \pm \Delta E$ such that 95% of future tests fall within that interval
- E_{test} is an unbiased estimate of $E = \textbf{true error rate}$
 - E = probability our system will misclassify a random instance
 - Take a random set of n instances, how many misclassified? (Our test set is one such set)
 - Flip E -biased coin n times, how many heads will we get?
 - Binomial distribution with mean = $n.E$, variance = $n.E.(1-E)$
 - $E_{future} = \#misclassified/n$, $\sim \text{Gaussian}$, mean E , variance = $E.(1-E)/n$
 - 2/3 future test sets will have error in $E \mp \sqrt{\frac{E(1-E)}{n}}$
 - $p\%$ confidence interval for future error

Confidence Interval for Future Error

- What range of errors can we expect for **future** test sets.
 - $E_{test} \pm \Delta E$ such that 95% of future tests fall within that interval
- E_{test} is an unbiased estimate of $E = \text{true error rate}$
 - E = probability our system will misclassify a random instance
 - Take a random set of n instances, how many misclassified? (Our test set is one such set)
 - Flip E -biased coin n times, how many heads will we get?
 - Binomial distribution with mean = nE , variance = $nE(1-E)$
 - $E_{future} = \text{\#misclassified}/n$, $\sim \text{Gaussian}$, mean E , variance = $E(1-E)/n$
 - 2/3 future test sets will have error in $E \mp \sqrt{\frac{E(1-E)}{n}}$
- $p\%$ confidence interval for future error
 - For $n=100$ examples, $p=0.95$ and $E=0.25$
 1. $\sigma = \sqrt{0.25 \times 0.75/100} = 0.043$
 2. $CI = 0.25 \pm 1.96 \cdot \sigma = 0.25 \pm 0.08$
 - For $n=100$ and $p=0.99$ and $E=0.25$, $CI = 0.25 \pm 0.11$
 - For $n=10000$ and $p=0.95$ and $E=0.25$, $CI = 0.25 \pm 0.008$



Training, Validation and Test Sets

- Training set, construct classifier
 - NB Count frequencies, DT: pick attributes to split on.
- Validation set: pick algorithm and adjustment settings
 - Pick best-performing algorithm (NB vs DT vs ...)
 - Fine tune the adjustments (tree depth, k in kNN, c in SVM (tbd))

Training, Validation and Test Sets

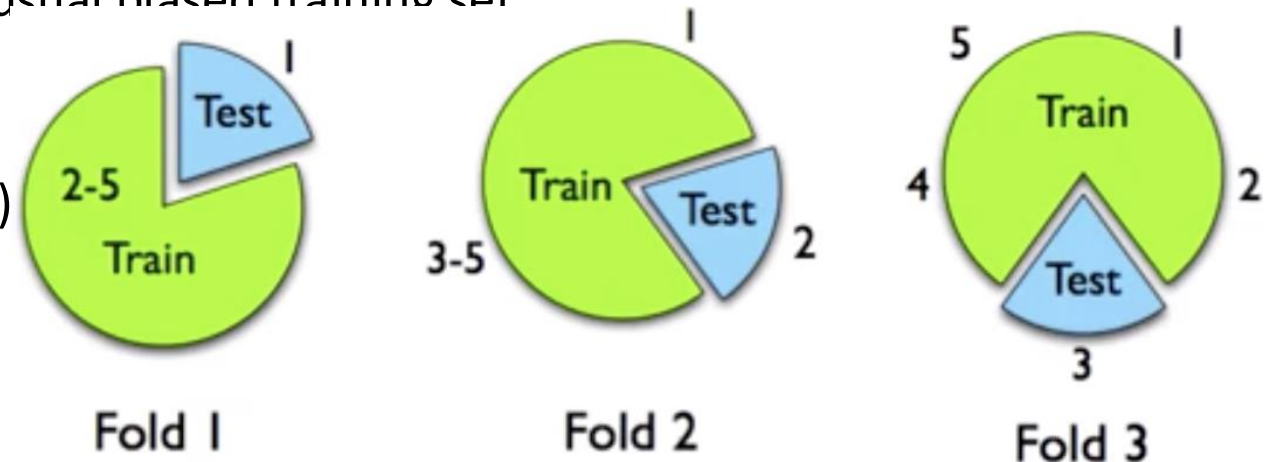
- Training set, construct classifier
 - NB Count frequencies, DT: pick attributes to split on.
- Validation set: pick algorithm and adjustment settings
 - Pick best-performing algorithm (NB vs DT vs ...)
 - Fine tune the adjustments (tree depth, k in kNN, c in SVM (tbd))
- Testing set: estimate future error rate
 - Never report best of any runs
 - Run only once, or report results of every run
- Split **randomly** to avoid bias
- Sometimes the use of the terms validation and testing datasets can be confused. A nice article commenting on this can be found here <https://machinelearningmastery.com/difference-test-validation-datasets/>
- Also (not on exam) you might be interested in this nice article which discusses the use of validation and learning curves for estimating how choices of hyperparameters affect under-vs over-fitting in your model development. https://scikit-learn.org/stable/modules/learning_curve.html

Cross Validation

- Conflicting priorities when splitting the dataset
 - Estimate future error as accurately as possible
 - Large testing set: big n_{test} leads to tight confidence interval
 - Learn classifier as accurately as possible
 - Large training set: big n_{train} leads to better estimates
 - Training and testing cannot overlap: $n_{\text{train}} + n_{\text{test}} = \text{const}$
- Idea: Evaluate Train-> Test, then Test->Train, average results

Cross Validation

- Conflicting priorities when splitting the dataset
 - Estimate future error as accurately as possible
 - Large testing set: big n_{test} leads to tight confidence interval
 - Learn classifier as accurately as possible
 - Large training set: big n_{train} leads to better estimates
 - Training and testing cannot overlap: $n_{\text{train}} + n_{\text{test}} = \text{const}$
- Idea: Evaluate Train-> Test, then Test->Train, average results
 - Every point is both training and testing, never at the same time
 - Reduces the chances of getting an unusual biased training set
- 5-fold cross-validation
 - Randomly split the data into 5 sets
 - Test on each in turn (train on 4 others)
 - Average the results over 5 folds
- More common is 10-fold



Leave-one-out

- N-fold cross validation (n = total number of instances)
 - Predict each instance, training on all $(n-1)$ instances
- Pros and Cons
 - Best possible classifier learned: $n-1$ training examples
 - High computational cost: relearn everything n times
 - Not an issue for instance-based methods like kNN
 - There are tricks to make such learning faster

Leave-one-out

- N-fold cross validation (n = total number of instances)
 - Predict each instance, training on all $(n-1)$ instances
- Pros and Cons
 - Best possible classifier learned: $n-1$ training examples
 - High computational cost: relearn everything n times
 - Not an issue for instance-based methods like kNN
 - There are tricks to make such learning faster
 - Classes not balanced in training/testing sets
 - No attributes, 2 equi-probable classes $\rightarrow$ wrong 100% of the time
 - Testing balance: {1 of A, 0 of B} vs. training: $\{n/2 \text{ of B}, n/2 - 1 \text{ of A}\}$

Stratification

- Problems with leave-one-out
 - training/testing data have classes in different proportions
 - Not limited to leave-one-out:
 - K-fold cross validation: random splits imbalance
- Stratification
 - Keep class labels balanced across training/testing sets
 - Simple way to guard against unlucky splits
 - Recipe:
 - Randomly split each class into K parts
 - Assemble i -th part from all classes to make the i -th fold

Nested Cross Validation

- see this <https://weina.me/nested-cross-validation/>
- And see <https://www.elderresearch.com/blog/nested-cross-validation>

Selecting Final Model

- Imagine using a nested cross validation procedure you are using an inner CV loop to determine which model is best to use on your data.
- The basic issue is **Given the evaluation of skill for your various models which one do you use?**
 - The one with the highest accuracy, lowest error etc? Right?
 - How do you know if that performance is genuinely representative of future data or is just a fluke? i.e. luck?
- One approach is to use statistical tests – hypothesis testing in particular.

Statistical Hypothesis Tests

- These tests focus on determining how likely it is to observe two samples (in this case machine learning algorithm performance results) given the assumption they are from the same distribution.
- This assumption that they are from the same distribution is called the *null hypothesis*.
- Statistical hypothesis tests are designed to allow us to determine whether or not to accept the null hypothesis given samples.

Applying Statistical Tests to Model Selection

- When selecting models we can use these statistical tests to determine if there is indeed a real difference between the performance results for the models. In other words
 - If given the performance results for the two models, the statistical test shows us that given these samples we cannot reject the null hypothesis then our observed difference in performance is down to chance. It is a fluke.
 - If on the other hand the performance results are such that when we apply the statistical tests that the null hypothesis can be rejected then the difference you are seeing in performance is genuine.
- NOTE: These statistical tests are probabilistic in output which means there is still a chance that what you are accepting as a fluke is not actually so and vice versa.

So what statistical hypothesis test should be used?

- It depends
- However lets consider the case where it is very difficult to run a model multiple times, e.g. a large deep learning model. This will reflect your assignments and other projects most likely.
- So McNemar's test is best in this case
 - Produces acceptable Type I error, i.e. false positives, i.e. that there is a significant difference between the two algorithms when in reality no such difference exists. Type II error on the other hand refers to failure to detect difference between two algorithms when there is one in reality (false negative).

McNemar's Test - Building the Contingency Table

Test datapoint	Classifier A Correct ?	Classifier B Correct?
1	Yes	No
2	No	No
3	No	Yes
4	No	No
5	Yes	Yes
6	Yes	Yes
7	Yes	Yes
8	No	No
9	Yes	No
10	Yes	Yes

This table is based on classifiers being trained on the exact same dataset and tested on the exact same testing instances

The Contingency Table

	Classifier B Correct	Classifier B Incorrect
Classifier A Correct	4	2
Classifier A Incorrect	1	3

In the case of the first cell in the table, we must sum the total number of test instances that Classifier A got correct and Classifier B got correct. For example, the first instance that both classifiers predicted correctly was instance number 5. The total number of instances that both classifiers predicted correctly was 4.

McNemar's Test

- Non-parametric (i.e. not assuming a particular distribution)
- Uses pairs of measurements (as per table)
- Checks to see if disagreements between the two algorithms match up. Does not consider if models are any good but only if they disagree in a similar way.
- $Statistic = \frac{(Yes/No - No/Yes)^2}{Yes/No + No/Yes}$

	Classifier B Correct	Classifier B Incorrect
Classifier A Correct	Yes/Yes	Yes/No
Classifier A Incorrect	No/Yes	No/No

McNemar's Test

	Classifier B Correct	Classifier B Incorrect
Classifier A Correct	Yes/Yes	Yes/No
Classifier A Incorrect	No/Yes	No/No

- The test statistic checks to see if there is a significant difference in the errors made by both models. It is checking to see if Yes/No and No/Yes boxes are similar counts. If not, then there is a difference in the error rate between the classifiers and therefore difference in performance.
- **Fail to Reject Null Hypothesis:** Classifiers have a similar proportion of errors on the test set.
- **Reject Null Hypothesis:** Classifiers have a different proportion of errors on the test set.

McNemar's Test

	Classifier B Correct	Classifier B Incorrect
Classifier A Correct	Yes/Yes	Yes/No
Classifier A Incorrect	No/Yes	No/No

- $p > \alpha$: fail to reject H_0 , no difference in the disagreement (e.g. treatment had no effect).
- $p \leq \alpha$: reject H_0 , significant difference in the disagreement (e.g. treatment had an effect).
- More detail (<https://machinelearningmastery.com/statistical-significance-tests-for-comparing-machine-learning-algorithms/>)
- Implementation of McNemars here - <https://machinelearningmastery.com/mcnemars-test-for-machine-learning/>

Training the final model

- The whole point of your cross validation and similar ideas is to figure out the appropriate model to use and the (hyper)parameter settings.
- Once this has been achieved you can ditch the model you developed using the training or cross validations processes and train your chosen model with chosen hyperparameters on the WHOLE dataset.

One last thing ... the No Free Lunch Theorem

- This is interesting. The next slide shows a paper you might like to read
– at least read the abstract!
- <https://www.kdnuggets.com/2019/09/no-free-lunch-data-science.html>

Do we Need Hundreds of Classifiers to Solve Real World Classification Problems?

Manuel Fernández-Delgado

MANUEL.FERNANDEZ.DELGADO@USC.ES

Eva Cernadas

EVA.CERNADAS@USC.ES

Senén Barro

SENEN.BARRO@USC.ES

CITIUS: Centro de Investigación en Tecnologías da Información da USC

University of Santiago de Compostela

Campus Vida, 15872, Santiago de Compostela, Spain

Dinani Amorim

DINANIAMORIM@GMAIL.COM

Departamento de Tecnologia e Ciências Sociais- DTCS

Universidade do Estado da Bahia

Av. Edgard Chastinet S/N - São Geraldo - Juazeiro-BA, CEP: 48.305-680, Brasil

Editor: Russ Greiner

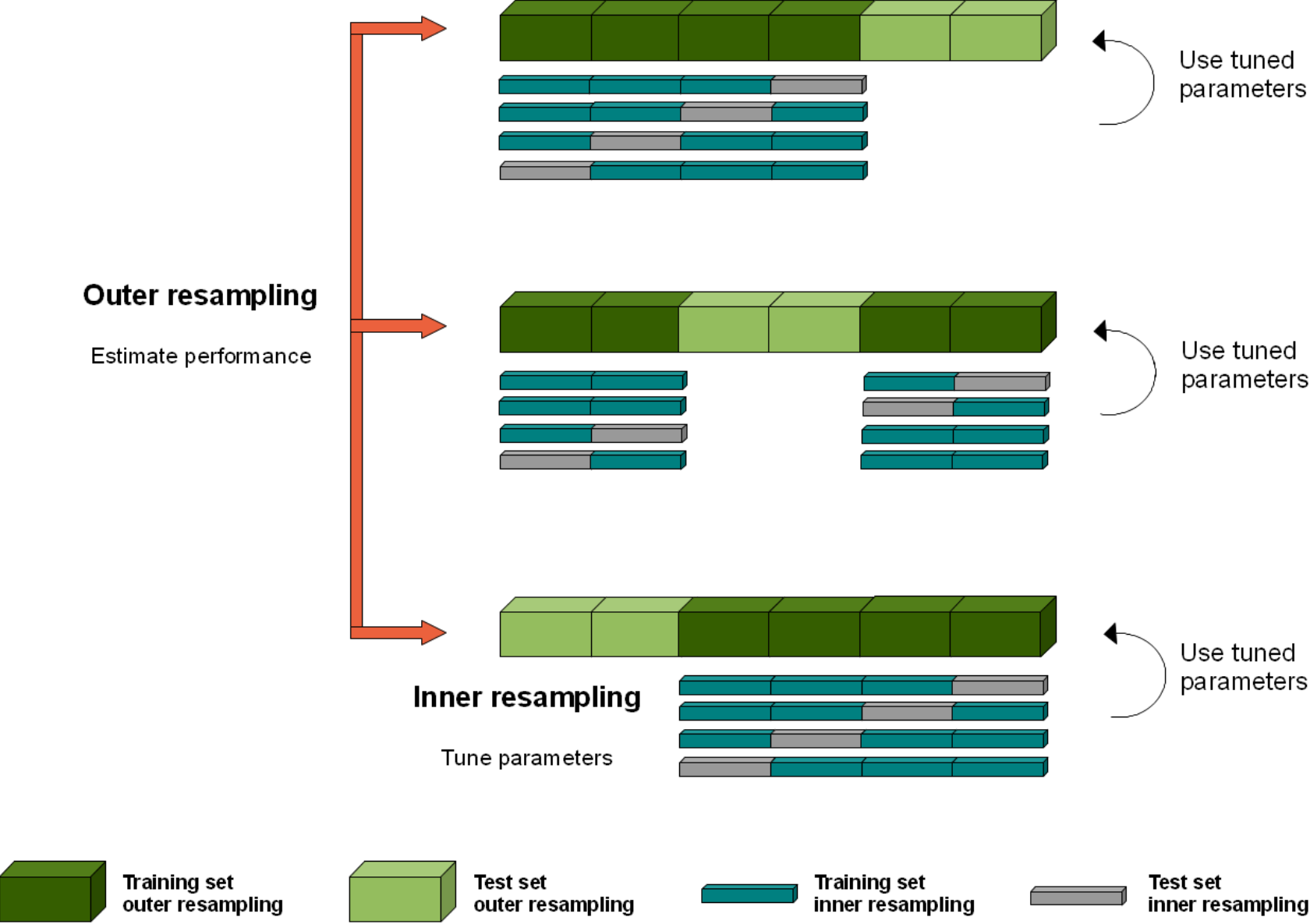
Abstract

We evaluate **179 classifiers** arising from **17 families** (discriminant analysis, Bayesian, neural networks, support vector machines, decision trees, rule-based classifiers, boosting, bagging, stacking, random forests and other ensembles, generalized linear models, nearest-neighbors, partial least squares and principal component regression, logistic and multinomial regression, multiple adaptive regression splines and other methods), implemented in Weka, R (with and without the caret package), C and Matlab, including all the relevant classifiers available today. We use **121 data sets**, which represent **the whole UCI** data base (excluding the large-scale problems) and other own real problems, in order to achieve significant conclusions about the classifier behavior, not dependent on the data set collection. **The classifiers most likely to be the bests are the random forest (RF)** versions, the best of which (implemented in R and accessed via caret) achieves 94.1% of the maximum accuracy overcoming 90% in the 84.3% of the data sets. However, the difference is not statistically significant with the second best, the SVM with Gaussian kernel implemented in C using LibSVM, which achieves 92.3% of the maximum accuracy. A few models are clearly better than the remaining ones: random forest, SVM with Gaussian and polynomial kernels, extreme learning machine with Gaussian kernel, C5.0 and avNNet (a committee of multi-layer perceptrons implemented in R with the caret package). The random forest is clearly the best family of classifiers (3 out of 5 bests classifiers are RF), followed by SVM (4 classifiers in the top-10), neural networks and boosting ensembles (5 and 3 members in the top-20, respectively).

- AFTER HERE NOT EXAMINABLE
BUT USEFUL

Nested Cross Validation – see this <https://weina.me/nested-cross-validation/>

ITEM 19



Nested Cross Validation

Students should look at <https://www.elderresearch.com/blog/nested-cross-validation>

What is the problem?

- Several scientific disciplines have been rocked by a crisis of reproducibility in recent years [1]. Not long ago, Bayer researchers found that they were only able to replicate 25% of the important pharmaceutical papers they examined [2], and an MIT report on Machine Learning papers found similar results. Some fields have begun to emerge from their crises, but other fields, such as psychology, may have not yet hit bottom [3] [4].

Why?

- We might imagine that this is because many scientists are good at science but not so adept with statistics. We might even imagine that we Analytics practitioners should have fewer problems because we are good at statistics. As a matter of fact, we find ourselves with an equivalent issue: predictive models that underperform once deployed. We have a powerful tool to prevent an underperforming model in Cross Validation (CV), but the ubiquity of CV in our modeling tools has led many Analysts to misunderstand how to properly use CV or appropriately create CV partitions, leading to lower-performing models.

Model Validation

- Before discussing Cross Validation, we need to understand why it is necessary. Model validation allows us to test the accuracy of our models, but it is difficult to do correctly. The most basic mistake an analytics team can make is to test a model on the same data that was used for training. Doing this is like giving a test to students after handing out the answers: any sufficiently flexible model will get a perfect score.

Partitioning Data

- A basic precaution that enables us to avoid testing and training on the same data is to divide our data into two sets before doing *any* analysis: one to train the model and the other to test the model. The training data represents the data that we will have in hand before deploying our model, and the test data represents the new, never-seen-before data that will arrive during production. This procedure is vital to modeling practice. Many who over-examine their total data learn too much to be surprised by the data subset later held out (unlike the real data of the future, which can always shock). They become overconfident in their models' accuracy until the cold cruel world brings ruin.



Bias vs Variance

- Though the hold-out sample approach is a very good practice, it has two main weaknesses. First, if the training or testing sample is unrepresentative of the whole, the model, or its test score, can be skewed. Careful sampling is advised. Second, if multiple models are being developed and compared, the train/test division is no longer good enough. This happens because we use the training data to construct each model, then use the test data to compare them. By definition, we will pick the model that does best on the test data — it's a relative comparison — but now that we've already peeked at that data, we're out of new data and have no way to test the winner to see how well it will do on data it's never seen. The model competition is actually part of training the winning model.

Note

- An important thing to note is that the term “multiple models” doesn’t simply mean very different models that use different techniques (like a random forest versus a neural network). If we have a logistic regression that we try *with* and *without* a particular predictor, or *with* and *without* taking the log of a variable, we have two different models.

- Practitioners familiar with mixture models or mixture-of-experts ensembles might see an analogy where the final model can be viewed as a mixture of the competing models, with the mixture coefficients having a single 1 (for the winner) and 0's for the models that lost. In this analogy, the “training” data is used to adapt the lower-level coefficients, and the “test” data is used to adapt the mixing coefficient, so we're left with no held-out data with which to test the resulting model.

Training, validation and testing partitions

- Seen this way, it is clear that when model selection is involved, we need three partitions: training, validation, and testing. Each model is trained on the training set, the models are compared with the validation* set, and the winner is tested on the test set to confirm how well it will perform on unseen data in the future. The concept of a three-way partition will be important to keep in mind as we move on to our discussion of cross validation: it still applies and can help our intuition as the process becomes more complex.

Cross Validation

- Cross validation (CV) takes the basic idea of a train/test partition and generalizes it into something more efficient and informative. In CV, we break the data up into K partitions and then, K times in turn, we select one partition for testing and use the remaining ones for training. It's more "fair" in its use of the data than a single train/test partition, since each case is used once in testing (and $K-1$ times in training), and, of course, never in both sets at once.

Cross Validation

- The magic of cross validation is that it provides us with an accuracy distribution rather than a point estimate. With 10-fold CV we obtain 10 accuracy measurements, which allows us to estimate a central tendency and a spread. The spread is often a critical piece of information, especially when making comparisons or choices. Never accept a point estimate when you can get a distribution instead. This addresses our first problem with a hold-out sample – that it will not be representative – since the distribution gives a better expected value for the accuracy, as well as a confidence (e.g., standard deviation) for that estimate.



Points regarding CV

- Cross validation is so ubiquitous that it often only requires a single extra argument to a fitting function to invoke a random 10-fold cross validation automatically. This ease of use can lead to two different errors in our thinking about CV:
- that using CV within our selection process is the same as *doing* our selection process via CV, or
- that randomly selecting CV partitions is inherently superior to other partitioning methods.
- Using CV properly and designing partitions strategically are important for analytics practitioners to understand.

Nested Cross Validation

- A chief confusion about CV is not understanding the need for multiple uses of it, within layers. For instance, one can use cross validation within the model selection process and a different cross validation loop to actually select the winning model. When we use CV in the model fitting process, typically by turning on an option in our regression or classification function, its repeated splitting is equivalent to a train/test partitioning, except more informative and efficient. As we've seen, a two-way split like this is not sufficient for model selection, which requires a three-way split.

Nested CV

- The CV equivalent to the training/validation/test split – that is, for model selection — requires that we run an inner CV that's equivalent to the train/validation partition within an outer CV that's equivalent to the validation/test partition. This is called Nested CV. If you hear someone say, “We selected our final model via CV,” it is essential to know whether they correctly used Nested CV or mistakenly used only an inner CV – a common mistake.

Nested CV

- Once we're comfortable with Nested CV, we should enlarge our vision of what building a model really entails to ensure a thorough modeling process. If we select certain variables but leave out others, that variable selection is actually part of our model. After all, we will select those same variables when we run our model in production to score new data. If we perform transformations on our selected variables, this is also part of our model, whether it shows up in a model's formula or not. Because of this, the outer CV should wrap around selection and transformation choices as well.

Choosing the right partitioning strategy

- The default in most analytics packages is to select CV partitions at random. This avoids certain partitioning errors — such as selecting the first part and latter lines in a file, which will be non-representative if the file is ordered — but a random result is not always the best answer. At times, it's actually the wrong answer.

- If we're doing classification, we need to guarantee that each class is represented in both training and test data, even if some classes are small. Purely random partitioning can end up with partitions that do not contain a particular class. When those partitions serve as the training set, CV breaks down, as the test data will contain a hitherto unseen class. If small classes are an issue, we need to use *stratified* random sampling – randomly breaking each class up into the partitions separately so that each class is represented proportionally in each partition.

CV for Time Series

- Similarly, if we're doing time series modeling, training and testing on a willy-nilly mixture of dates will break the sequential nature of a time series. Going back to our analogy where training data represents the data we have and testing data represents data from the future, we need to make sure that when each partition is used for testing, only samples from earlier in time are used for training. One way to do this is to partition by year/month and then step through each partition, only using earlier partitions for training.

- This idea of purposefully partitioning is necessary for time series, but it can be illuminating in other cases as well. For example, we could partition our data by city to see how our model works for a new city we've never seen before, or on large cities if we've only seen small cities before. Different kinds of partitions ask different questions of our model, giving us a feel for how it may perform in varied circumstances; that knowledge is the key to not being unpleasantly surprised by our model's performance in production.

Summary

- Models that underperform our expectations break trust with our clients — even a good model is embarrassing if a great model was anticipated. One of the key techniques for avoiding the pain of an under-performing model is cross validation, properly applied.
- When using CV, we must know when single CV or Nested CV is required. If your analytics tool of choice does not directly support Nested CV, extra effort will be needed. Further, we need to consider how large our model building process actually is so that we can appropriately wrap it in CV. Finally, random partitioning is a default for most CV commands; make sure it is appropriate for your case.